



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Locators

APRIL, 2026



Contents

- Web Inspector
- Locators
- Types of Locators

Web Inspector



What is Web/Browser Inspector

- a web inspector or a browser inspector helps to identify/locate what's inside the web page in a browser.
- The inspector shows the CSS, HTML, and JavaScript of the web page so that the developer or the tester can analyze the web page and use the browser developer tools' features to run a few checks.
- Example:
 - Checking out the color of some text on the web page.
 - For this, I can change the color code within the browser through the web inspector on that particular line and see live on the web page.
- **These changes done on the web inspector are temporary and will disappear once we close the tab.**

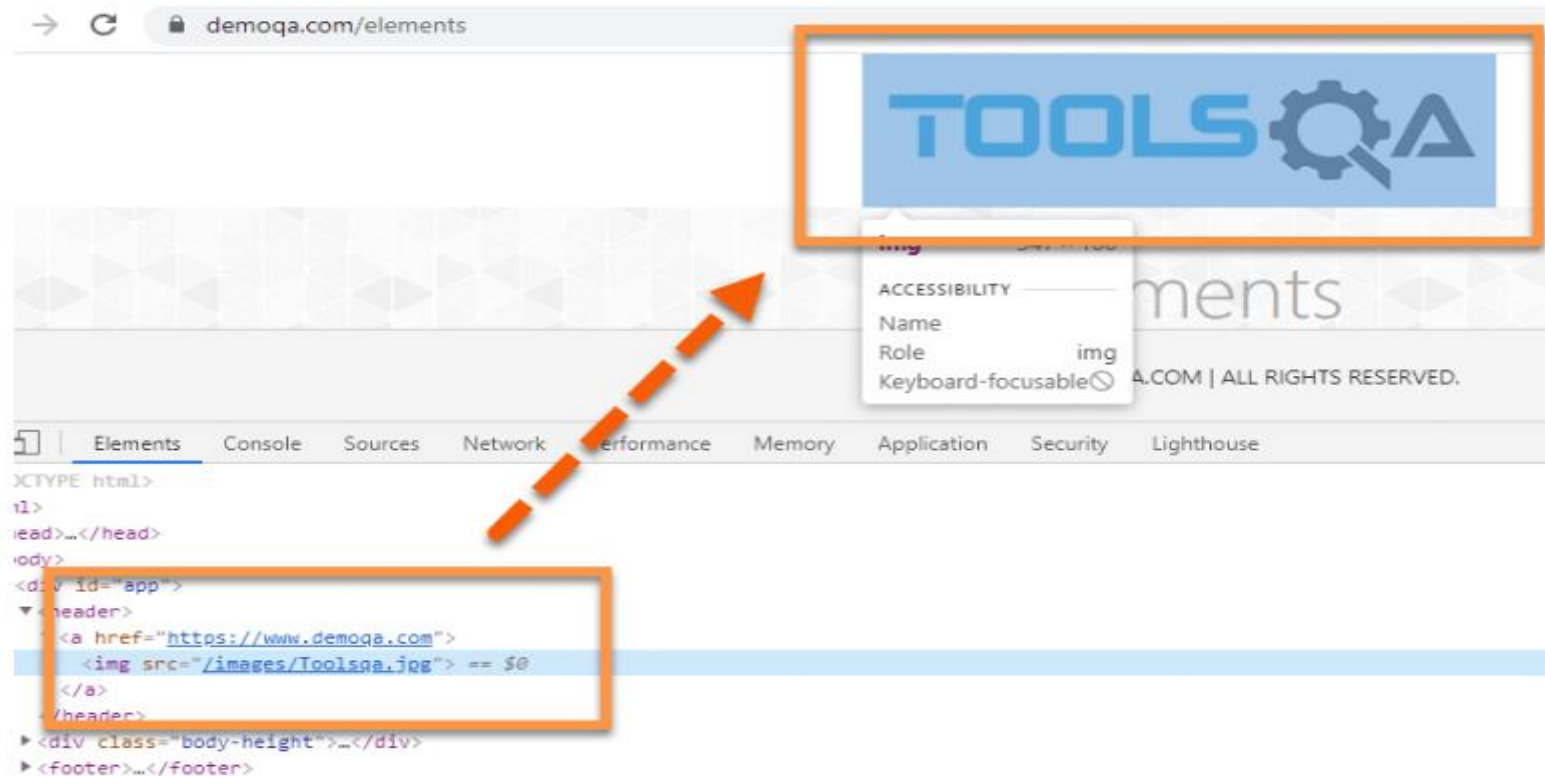
Inspecting Web Browser Elements

- We can open up the web Inspector using the following options:
 - 1) Right Click on the Web page and Select Inspect.
 - 2) Press Ctrl+Shift+C.
- How to inspect elements in Google Chrome?
 - 1) Launch the Google Chrome web browser and open up the inspect panel by pressing F12 or the methods described above (right-click -> inspect).
 - 2) Once you open up the inspect panel, you would notice that there are multiple tabs available on the top, such as Elements, Console, Sources, Network, and Performance. However, the Elements tab is the one which consists of all the HTML properties belonging to the current web page.

Web Inspector

Inspecting Web Browser Elements

3) Now, if we hover the mouse pointer over the HTML tags in the DOM, it will highlight the corresponding elements it represents on the webpage.



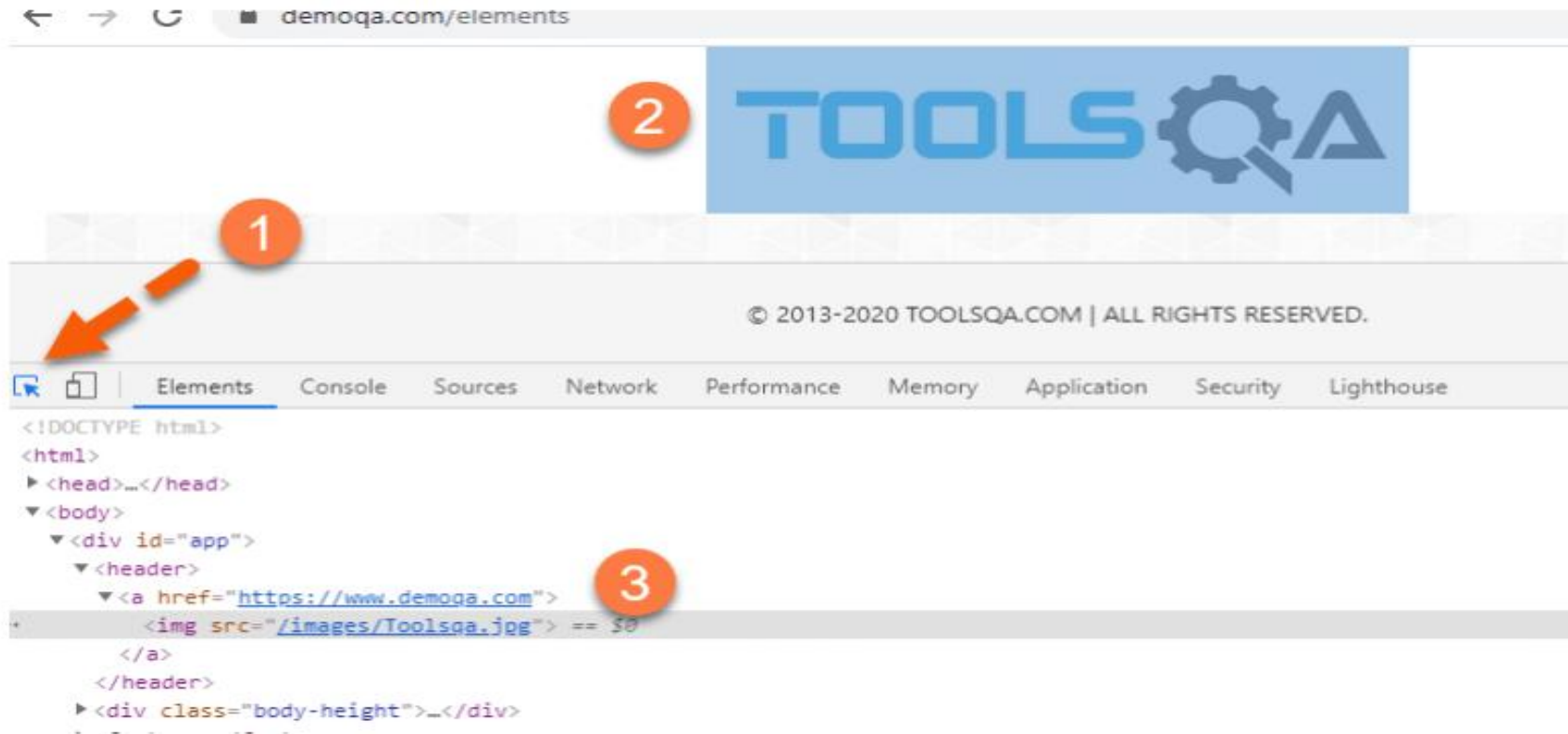
Inspecting Web Browser Elements

4) Now, the main point is, how do we find the web element in the DOM.

- Click on the "Mouse Icon" arrow (as designated by Marker 1 in below screenshot) and then select the web element on the web page.
- It will automatically highlight the corresponding HTML element in the DOM.
- Suppose we want to find the HTML elements corresponding to the banner image (as shown below by marker 3).
- When we select the mouse point and click on the banner image, it will automatically highlight the corresponding HTML element, as shown by marker 2

Web Inspector

Inspecting Web Browser Elements



Locators



Locators

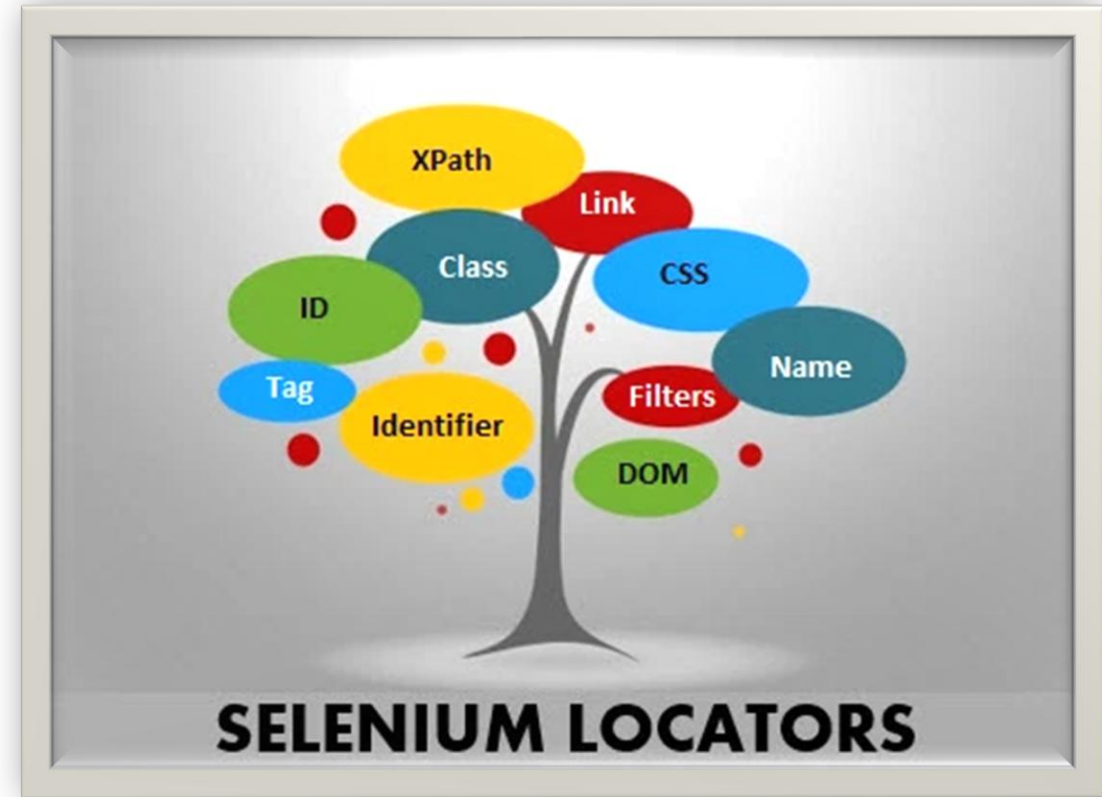
Introduction to Locators

- Identification of the correct GUI element on a web page is pre-requisite for creating any successful automation script.
- Locating an element is one of the most important tasks in selenium tests but there is no fixed strategy that you can follow to identify an element (object) uniquely on the web page.
- Locating any element on a web page is complex task.
- It totally depends on the user's ideas and their comfort level in locating web elements.
- To recognize the element (Object) uniquely on the web page, Selenium gives us some locator strategies.

Locators

What is Locator?

- Locator in selenium is like an address that is used to identify the web element uniquely on the web page.
- In other words, Locator is a unique command used to identify web elements placed within a web page.
- And almost all UI automation tools provide the capability to use locators for the identification of HTML elements on a web page.
- It is very important to use the correct locator for quick execution testing.



Types of Locators

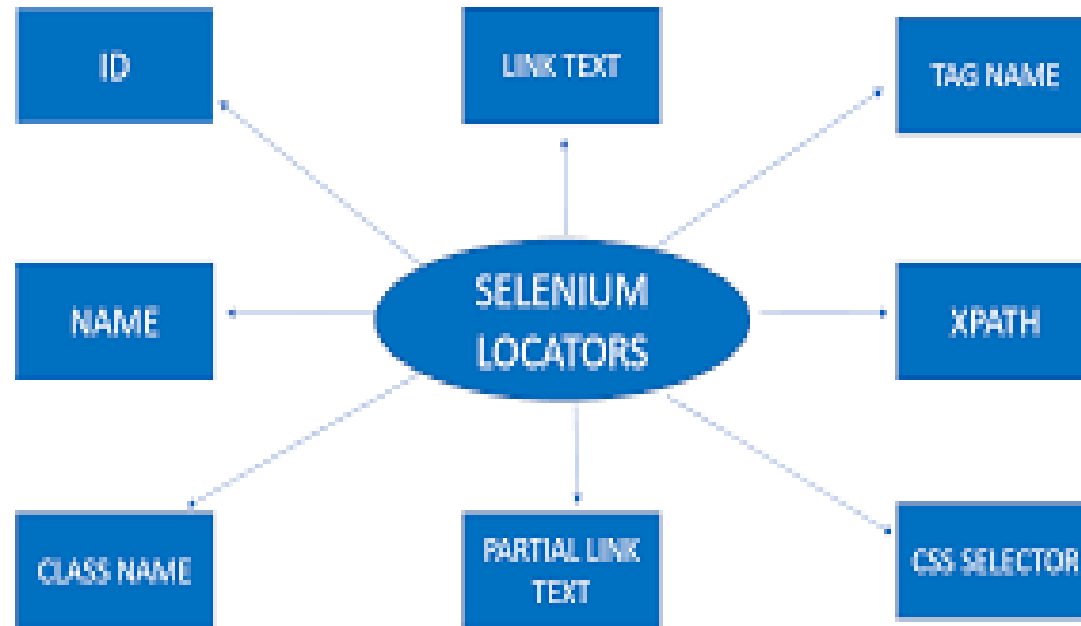


Types of Locators

Selenium Locators Types

- There are various types of locators, using which we can identify a web element uniquely on the Webpage.
- Selenium supports the following locators:

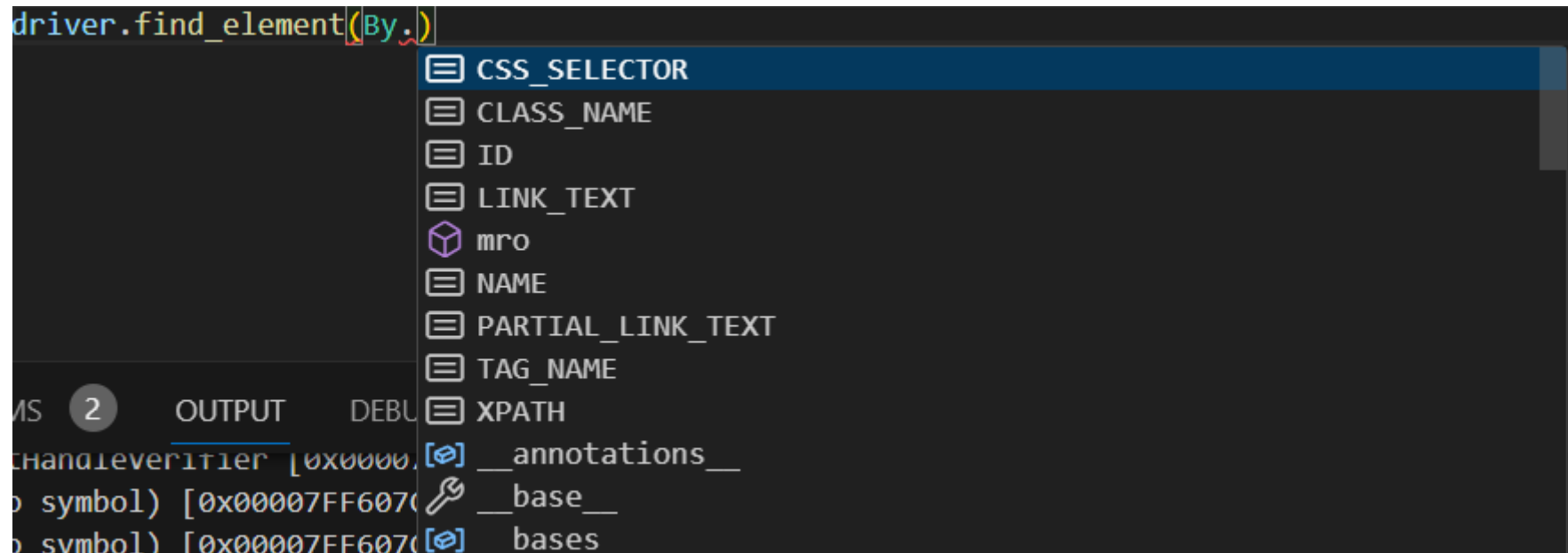
1. Id
2. Name
3. Class Name
4. HTML Tag Name
5. Linked Text
6. Partial Link Text
7. Xpath
8. CSS



Types of Locators

Selenium Locators Types

- You can quickly identify all the supported locators by Selenium by browsing all the visible methods on the "By" class, as shown below:



Locators

Id

- ID locator is the safest and fastest locator to find the location of an element based on the value of “ID” attribute on a web page.
- Since ID is unique for each element on the web page so it can be easily identified. It is always the first choice for testers. It is mostly an account number, college name, username, password, or sign in button that will be unique.
- The general syntax to use id locator is as follows:

Element = driver.find_element(By.ID, value=“Attribute value”)

where,

driver ➡ Object reference variable of the WebDriver object.

element ➡ Variable used to store the returning web element.

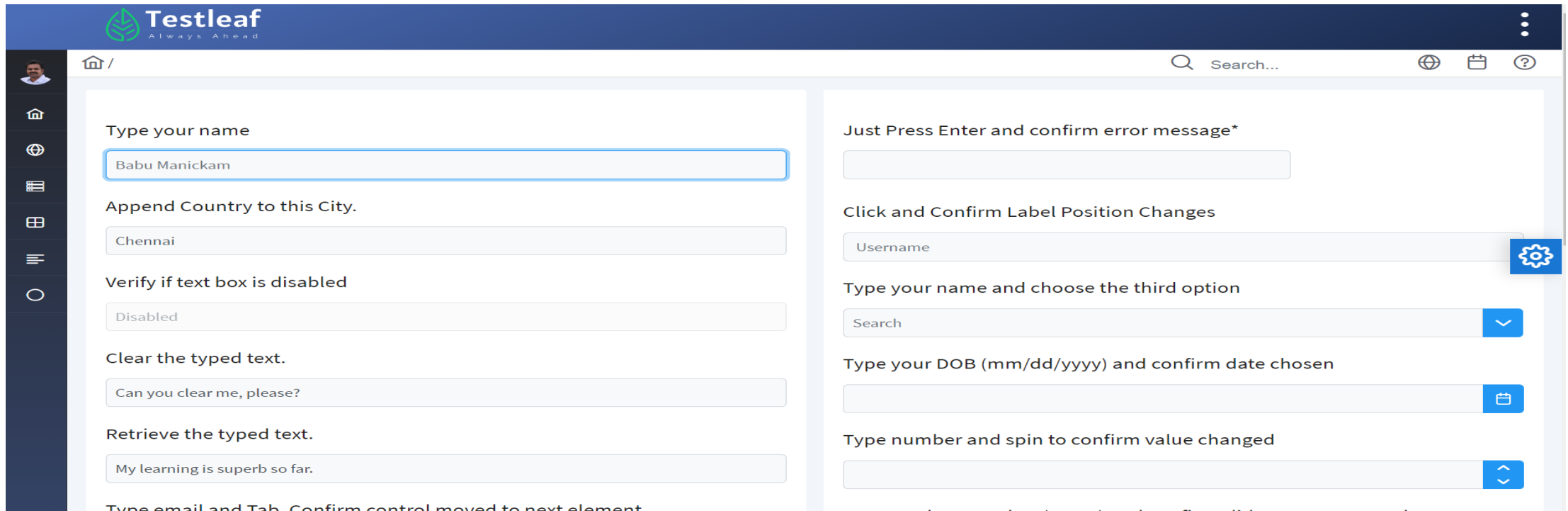
By ➡ By is a class which provides a mechanism that is used to locate elements within a document.

- Example: **Element = driver.find_element(By.ID, value=“Emp01”)**

Locators

Id Example

- Launch google chrome browser and navigate to the following testing website Url “https://www.leafground.com/input.xhtml” you can see the text box **Type your name**.
- We will locate the Type your name text box with the help of Id Locator.



The screenshot shows the Testleaf website interface. The header includes the Testleaf logo and a search bar. A sidebar on the left contains navigation icons. The main content area is divided into two columns. The left column contains several text input fields with labels: 'Type your name' (containing 'Babu Manickam'), 'Append Country to this City.' (containing 'Chennai'), 'Verify if text box is disabled' (containing 'Disabled'), 'Clear the typed text.' (containing 'Can you clear me, please?'), 'Retrieve the typed text.' (containing 'My learning is superb so far.'), and 'Type email and Tab. Confirm control moved to next element'. The right column contains: 'Just Press Enter and confirm error message*' (empty), 'Click and Confirm Label Position Changes' (containing 'Username' with a settings icon), 'Type your name and choose the third option' (containing 'Search' with a dropdown arrow), 'Type your DOB (mm/dd/yyyy) and confirm date chosen' (empty with a calendar icon), and 'Type number and spin to confirm value changed' (empty with a spin icon).

Locators

Id Example

- You will notice that there are input tag, class, name and id. Use Id `j_idt88:name` to locate the text box element Type your name.



The screenshot shows the Testleaf application interface. On the left, a sidebar contains navigation icons. The main content area has three sections: 'Type your name' with a text input containing 'Babu Manickam', 'Append Country to this City.' with a text input containing 'Chennai', and 'Verify if text box is disabled' with a disabled text input containing 'Disabled'. On the right, there are three more sections: 'Just Press Enter and confirm error message*' with an empty input, 'Click and Confirm Label Position Changes' with a text input containing 'Username', and 'Type your name and choose the third option' with a dropdown menu showing 'Search'.

The browser developer tools are open at the bottom. The 'Elements' tab shows the following HTML structure:

```
<input type="hidden" name="j_idt88" value="j_idt88">
<h5>Type your name</h5>
<div class="grid formgrid">
  <div class="col-12">
    <input id="j_idt88:name" name="j_idt88:name" type="text" class="ui-inputfield ui-inputtext ui-widget ui-state-default ui-corner-all" placeholder="Babu Manickam" role="textbox" aria-readonly="false" aria-disabled="false"> == $0
  </div>
</div>
```

The 'Styles' tab on the right shows the default styles for the selected element, including the class `ui-inputfield` and `ui-inputtext`.

C# Selenium Program – Id Locator

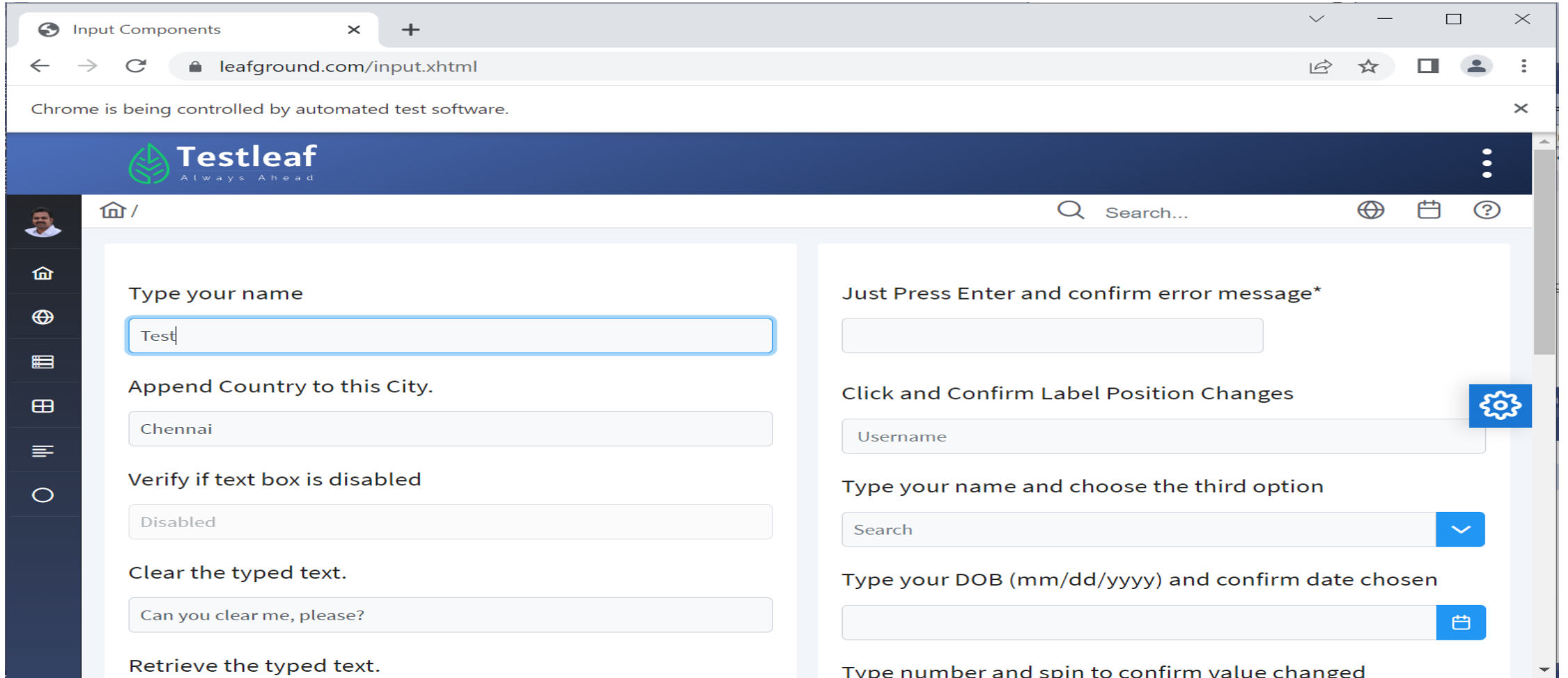
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.leafground.com/input.xhtml")

#1. Using ID Locator to locate Text Area Field
driver.find_element(By.ID, value="j_idt88:name").send_keys("Test")

time.sleep(3)
```

C# Selenium Program – Id Locator - Output



The screenshot displays a web browser window with the address bar showing "leafground.com/input.xhtml". The page title is "Input Components". A notification at the top states "Chrome is being controlled by automated test software." The Testleaf logo is visible in the top left corner. The main content area is divided into two columns of input components. The left column includes a text box with "Test", a text box with "Chennai", a disabled text box with "Disabled", a text box with "Can you clear me, please?", and a text box with "Retrieve the typed text.". The right column includes a text box with "Just Press Enter and confirm error message*", a text box with "Click and Confirm Label Position Changes", a text box with "Username", a text box with "Type your name and choose the third option", a text box with "Search", a text box with "Type your DOB (mm/dd/yyyy) and confirm date chosen", and a text box with "Type number and spin to confirm value changed". A blue gear icon is visible on the right side of the page.

Input Components

leafground.com/input.xhtml

Chrome is being controlled by automated test software.

Testleaf
Always Ahead

/

Search...

Type your name

Test

Append Country to this City.

Chennai

Verify if text box is disabled

Disabled

Clear the typed text.

Can you clear me, please?

Retrieve the typed text.

Just Press Enter and confirm error message*

Click and Confirm Label Position Changes

Username

Type your name and choose the third option

Search

Type your DOB (mm/dd/yyyy) and confirm date chosen

Type number and spin to confirm value changed

Locators

Name

- Every element in the webpage has many attributes, and a Name is one among them. It is a very efficient strategy to locate an element by name.
- Name locator is the second safest and fastest locator to locate an element based on the value of “name” attribute on the web page. The name cannot be unique for each element at all times.
- If there are multiple elements with the same name on a web page then Selenium will always perform the action on the first matching element.
- Username and Password fields are examples that can be identified with “name” attribute. The general syntax to use name locator is as follows:

Locators

Name

- The general syntax to use name locator is as follows:

Element = driver.find_element(By.NAME, value="Attribute value")

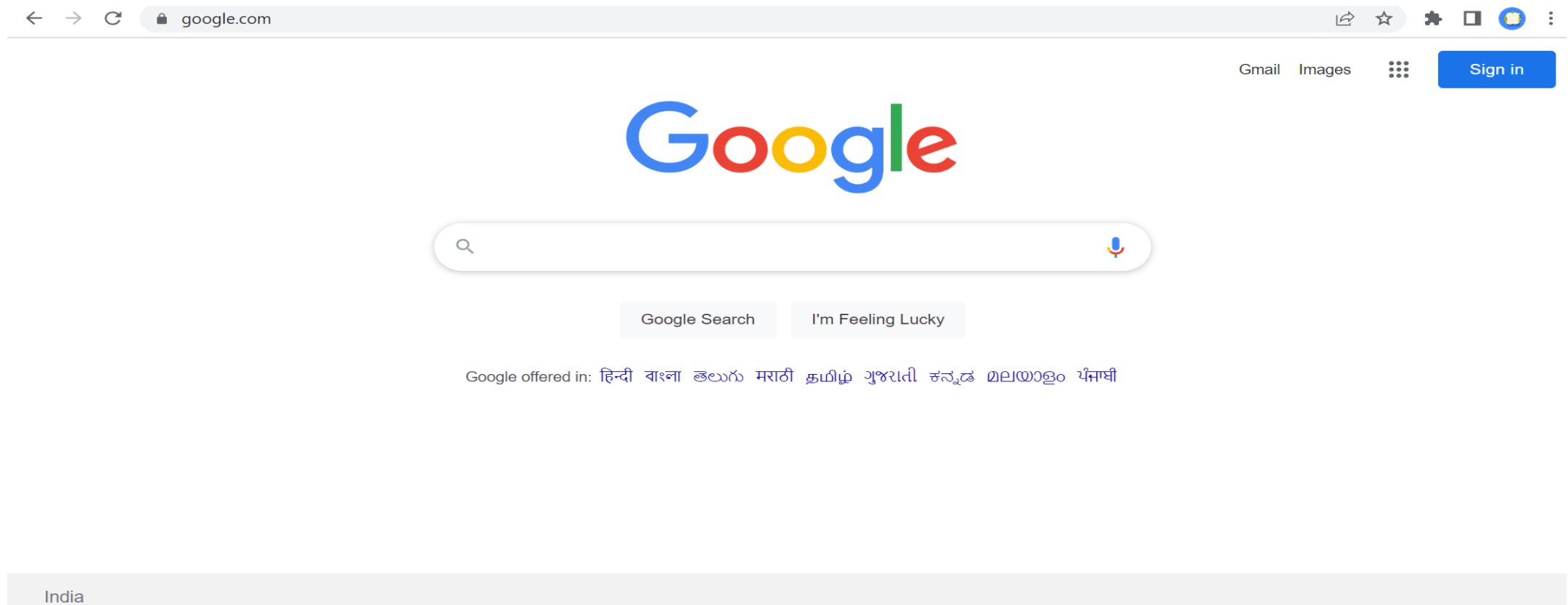
- Example

Element = driver.FindElement(By.NAME, value="username")

Locators

Name Example

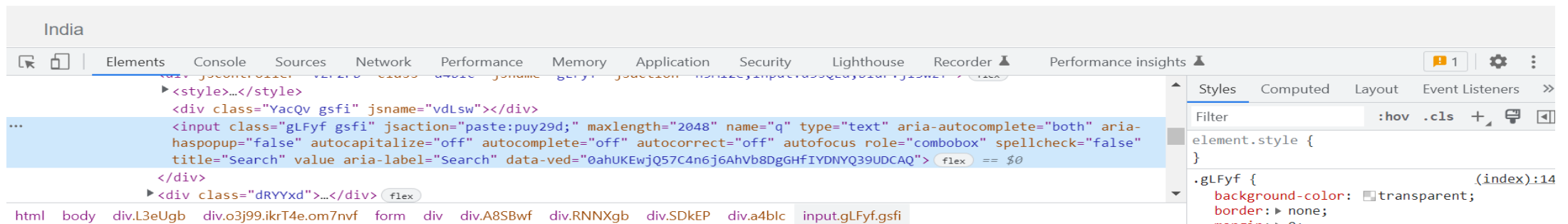
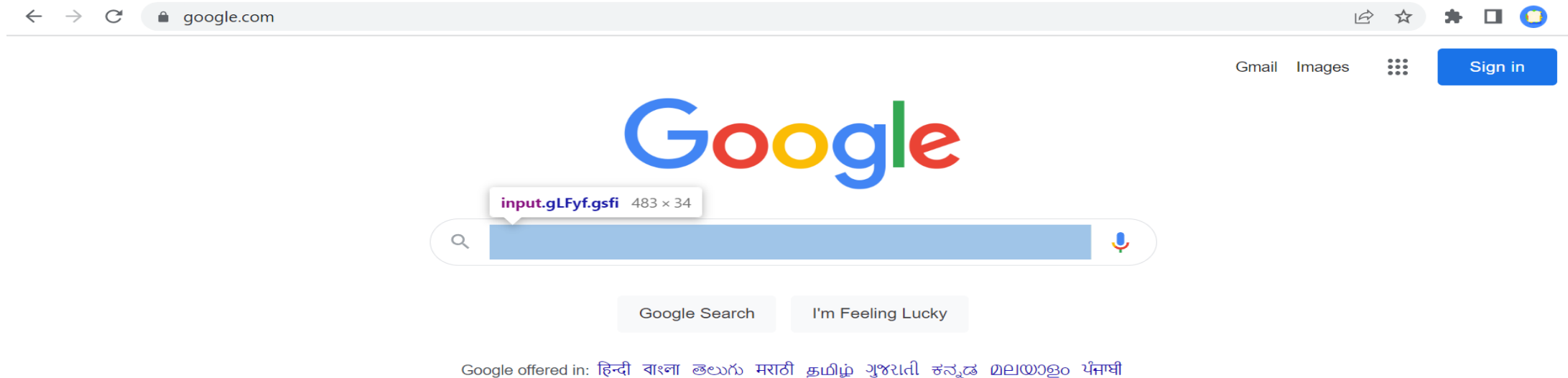
- Launch google chrome browser and navigate to google website. You can see search text box.
- Now will locate the search text box with the help of Name Locator.



Locators

Name Example

- You will notice that there are input tag, class, name and id. Use q to locate the search text box.



C# Selenium Program – Name Locator

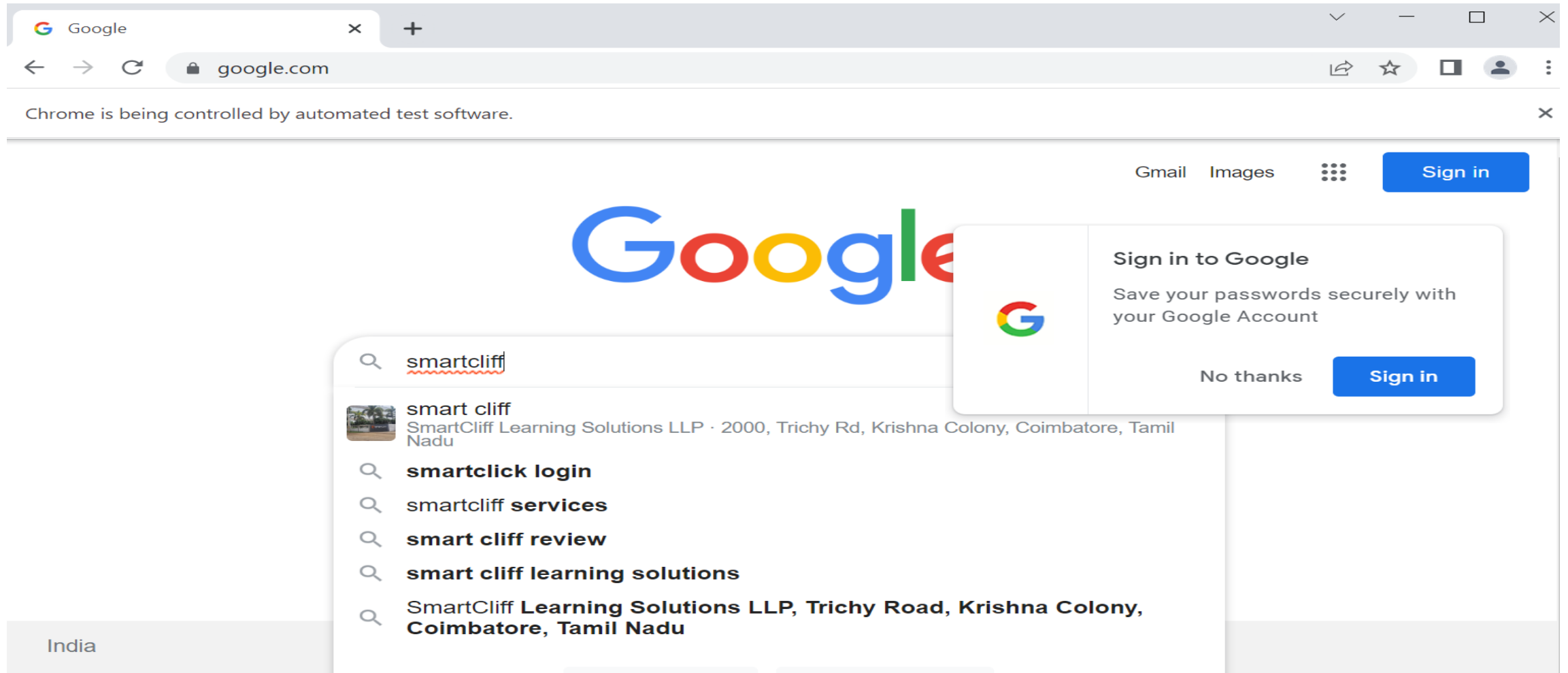
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.google.com")

#2. Using NAME Locator
driver.find_element(By.NAME, value="q").send_keys("smartcliff")

time.sleep(3)
```

C# Selenium Program – Name Locator - Output



Locators

Class Name

- ClassName allows to find web elements based on the class values of the DOM.
- For example, if we want to identify or perform any operation on the form element, we can use the class to identify it.
- To identify it successfully, we need to make sure that the class name value that we are using for locating the web element, i.e., web form, is unique, and any other class doesn't have the same value.
- If any other class contains the same value as this, then Selenium will select whichever element comes first while scrapping through the web page.
- The general syntax to use id locator is as follows:

Element = driver.find_element(By.CLASS_NAME, value="Attribute value")

Locators

ClassName Example

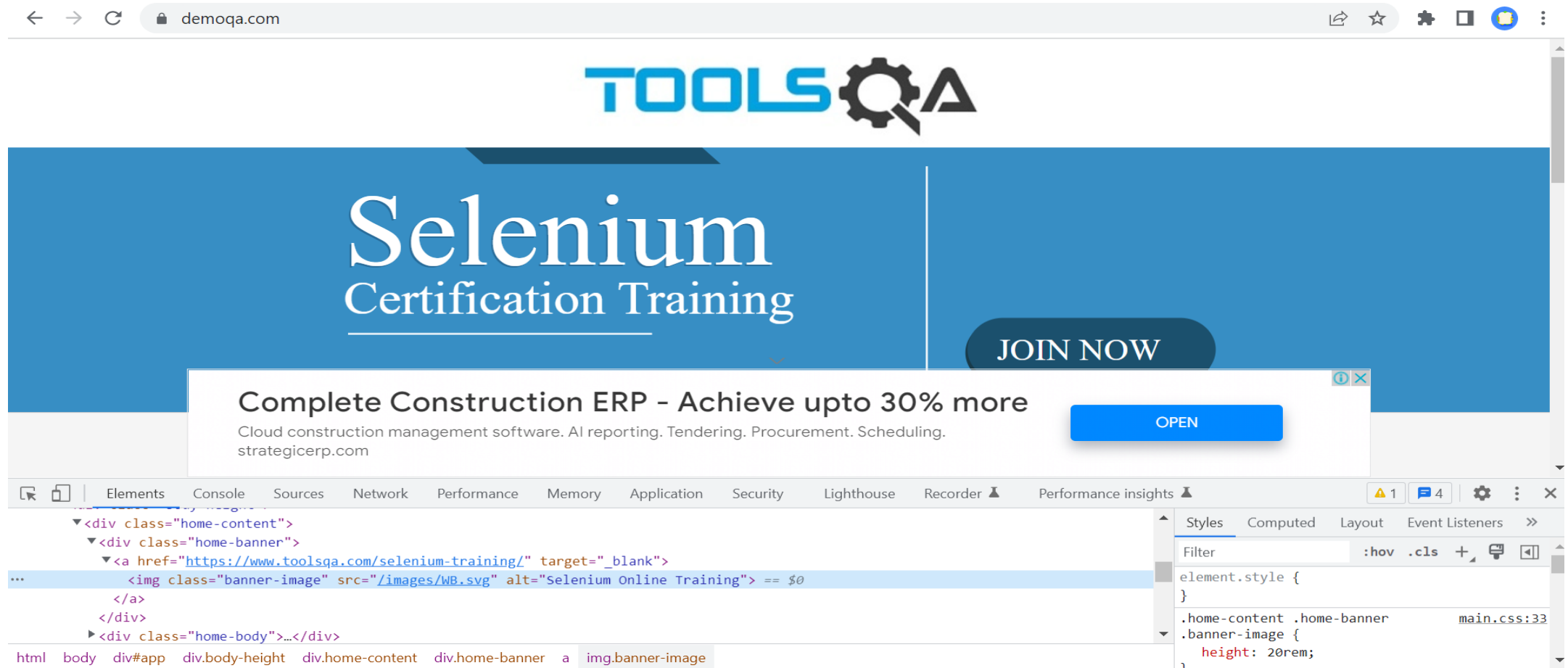
- Launch google chrome browser and navigate to toolsqa website <https://demoqa.com/>. You can see banner image.
- Now will locate the search text box with the help of ClassName Locator.



Locators

ClassName Example

- You will notice that the form image tag with class = “banner-image”.



The screenshot shows a web browser at `demoqa.com` displaying a banner for "Selenium Certification Training". The banner features the text "Selenium Certification Training" and a "JOIN NOW" button. Below the banner, there is a promotional message for "Complete Construction ERP - Achieve upto 30% more" with an "OPEN" button.

The browser's developer tools are open, showing the HTML structure. The selected element is an `img` tag with the class `banner-image`. The HTML structure is as follows:

```

<div class="home-content">
  <div class="home-banner">
    <a href="https://www.toolsqa.com/selenium-training/" target="_blank">
       == $0
    </a>
  </div>
</div>
<div class="home-body">...</div>
  
```

The CSS styles for the selected element are shown in the right pane:

```

element.style {
}
.home-content .home-banner
.banner-image {
  height: 20rem;
}
  
```

C# Selenium Program – ClassName Locator

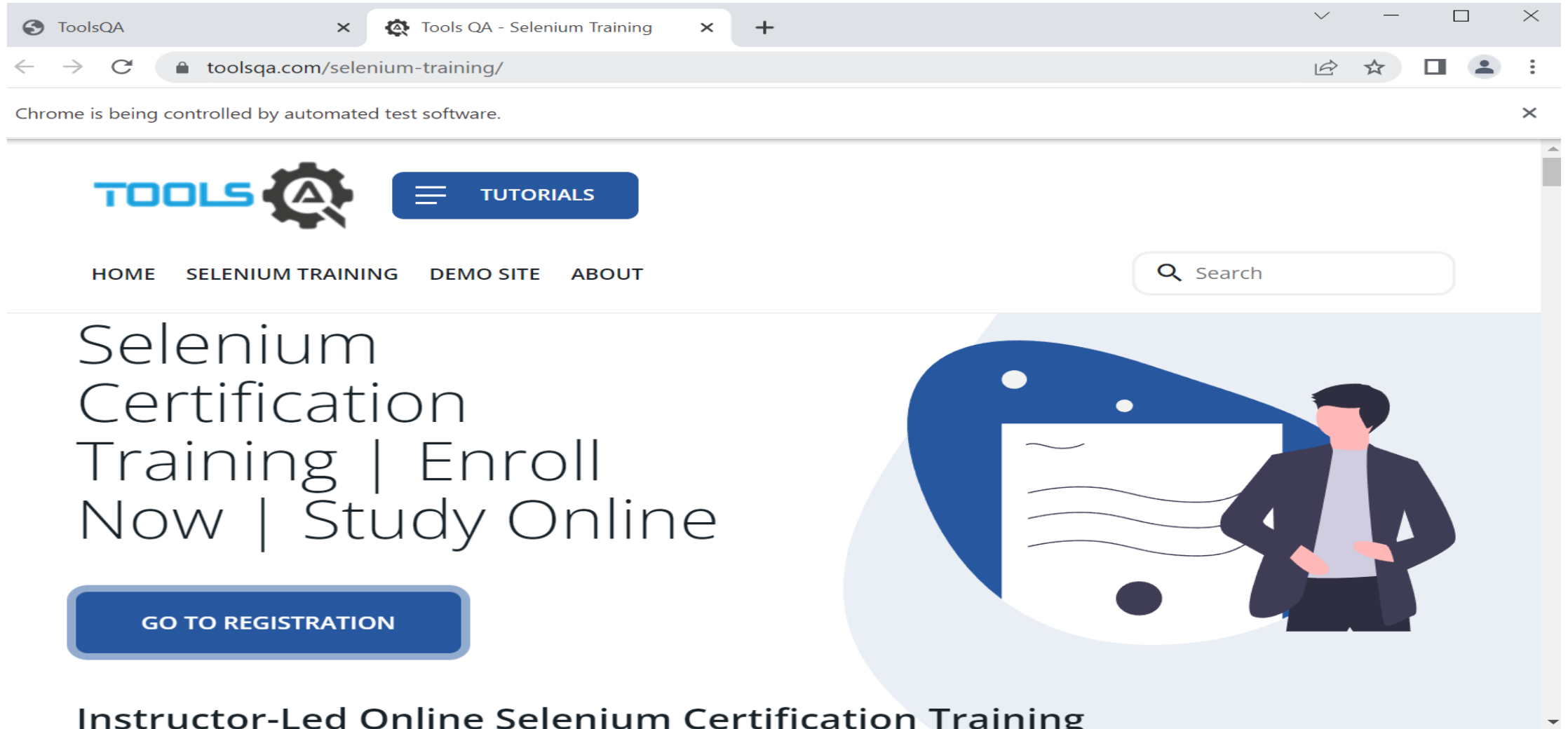
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.demoqa.com")

#3. Using Class Name Locator
driver.find_element(By.CLASS_NAME, value="banner-image").click()

time.sleep(3)
```

C# Selenium Program – ClassName Locator - Output



Locators

TagName

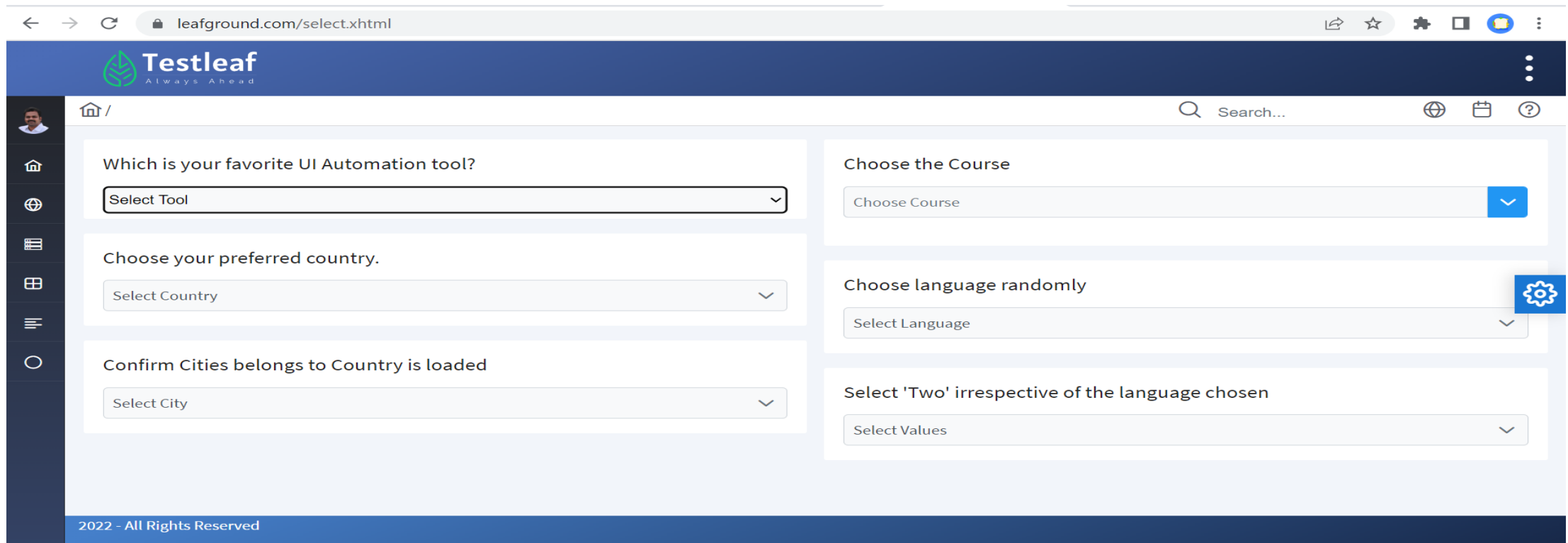
- This locator type uses tag names to identify elements in a web page. The tag name is the HTML tag, such as `input`, `div`, `anchor tag`, `button`, etc.
- TagName locator is used to finding an element from the group elements like checkboxes, drop-downs, etc. It locates an element by its tag name.
- The same tag will be present multiple times on a web page, so it's advisable to use `findElements()` method when using the TagName locator.
- The general syntax to use id locator is as follows:

`Element = driver.find_element(By.TAG_NAME, value="Attribute value")`

Locators

TagName Example

- Launch google chrome browser and navigate to <https://www.leafground.com/select.xhtml>. You can see select tool drop-down list.
- Now will locate the select tool drop-down list with the help of TagName Locator.



The screenshot shows a web browser window with the URL [leafground.com/select.xhtml](https://www.leafground.com/select.xhtml). The page features the Testleaf logo and a sidebar with navigation icons. The main content area contains several form elements:

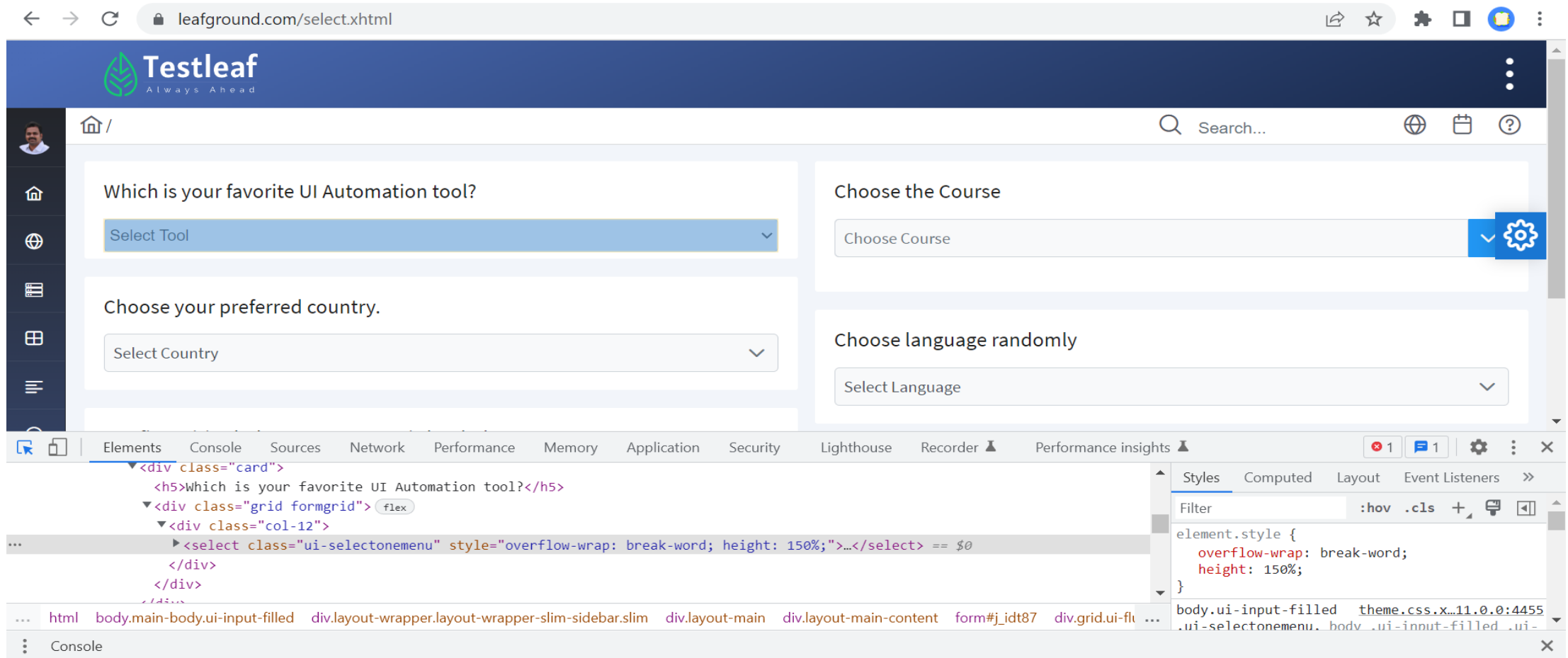
- Which is your favorite UI Automation tool?**: A dropdown menu with the placeholder text "Select Tool".
- Choose your preferred country.**: A dropdown menu with the placeholder text "Select Country".
- Confirm Cities belongs to Country is loaded**: A dropdown menu with the placeholder text "Select City".
- Choose the Course**: A dropdown menu with the placeholder text "Choose Course".
- Choose language randomly**: A dropdown menu with the placeholder text "Select Language".
- Select 'Two' irrespective of the language chosen**: A dropdown menu with the placeholder text "Select Values".

The footer of the page displays "2022 - All Rights Reserved".

Locators

TagName Example

- You will notice that the select tool drop-down list with tag = “select”.



The screenshot shows a web browser at `leafground.com/select.xhtml` displaying a form titled "Testleaf Always Ahead". The form contains several interactive elements:

- "Which is your favorite UI Automation tool?" with a dropdown menu labeled "Select Tool".
- "Choose the Course" with a dropdown menu labeled "Choose Course".
- "Choose your preferred country." with a dropdown menu labeled "Select Country".
- "Choose language randomly" with a dropdown menu labeled "Select Language".

The browser's developer tools are open, showing the HTML structure of the first dropdown. The HTML snippet is as follows:

```
<div class="card">
  <h5>Which is your favorite UI Automation tool?</h5>
  <div class="grid formgrid">
    <div class="col-12">
      <select class="ui-selectonemenu" style="overflow-wrap: break-word; height: 150%;">...</select> == $0
    </div>
  </div>
</div>
```

The "Styles" panel on the right shows the following styles for the selected element:

```
element.style {
  overflow-wrap: break-word;
  height: 150%;
}
```

The breadcrumb at the bottom of the developer tools indicates the element's path: `html > body > main > body.ui-input-filled > div.layout-wrapper.layout-wrapper-slim-sidebar.slim > div.layout-main > div.layout-main-content > form#j_idt87 > div.grid.ui-fl...`

C# Selenium Program – TagName Locator

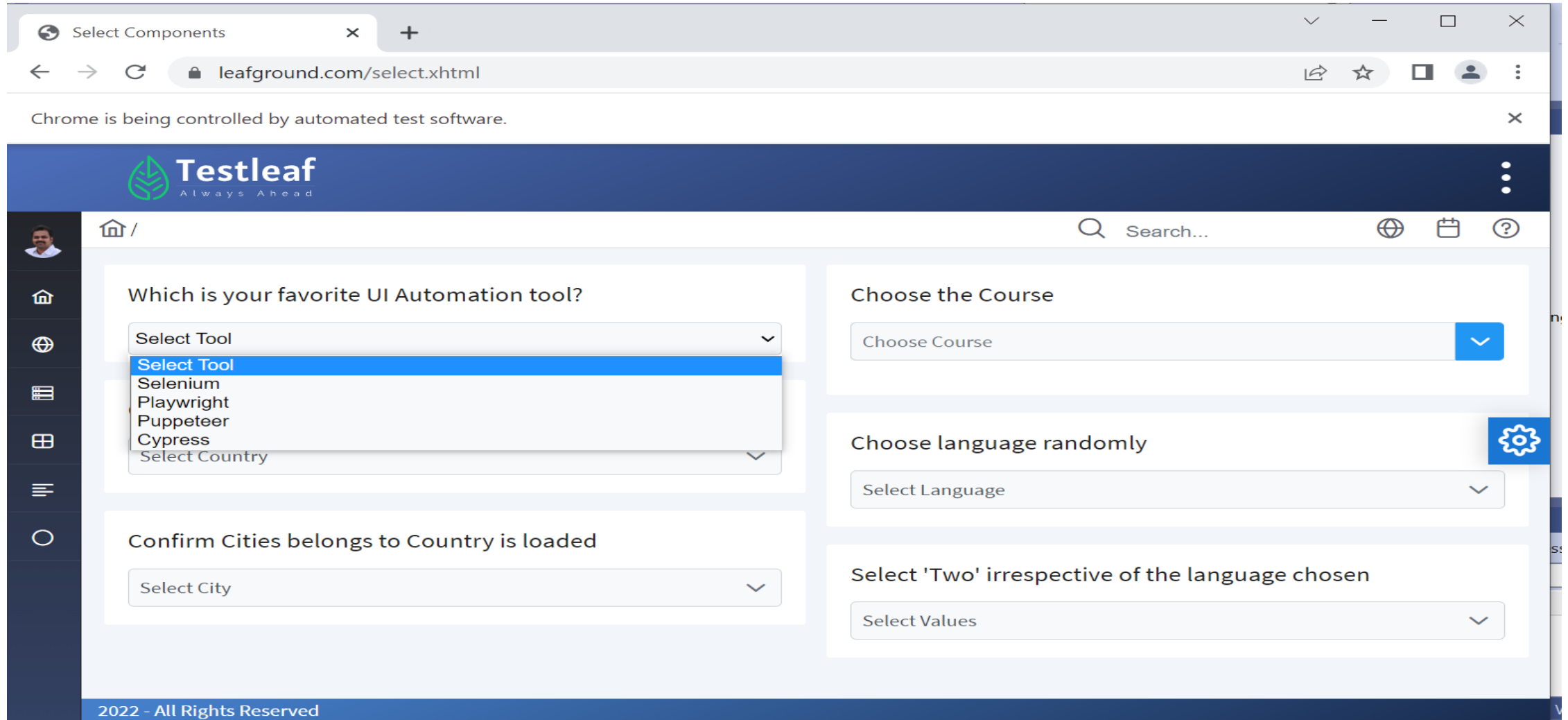
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.leafground.com/select.xhtml")

#4. Using Tag Name Locator
driver.find_element(By.TAG_NAME, value="select").click()

time.sleep(3)
```

C# Selenium Program – TagName Locator - Output



Locators

LinkText

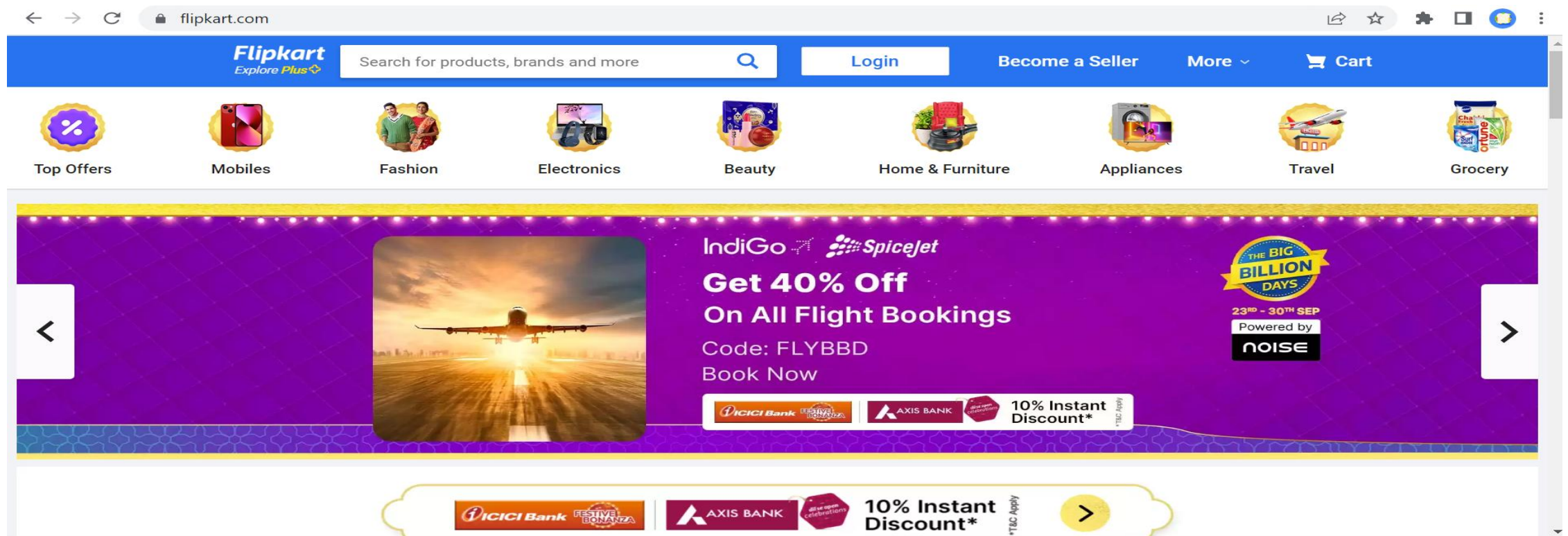
- This locator finds an element by the link text. If there is one unique element on the web page, you can easily find an element with the link text.
- The LinkText helps you in identifying the hyperlinks on a webpage. You must use anchor tag (<a>) for completing this action. If you want to create hyperlinks you must use both anchor tags and then LinkText.
- The general syntax to use id locator is as follows:

Element = driver.find_element(By.LINK_TEXT, value="Attribute value")

Locators

LinkText Example

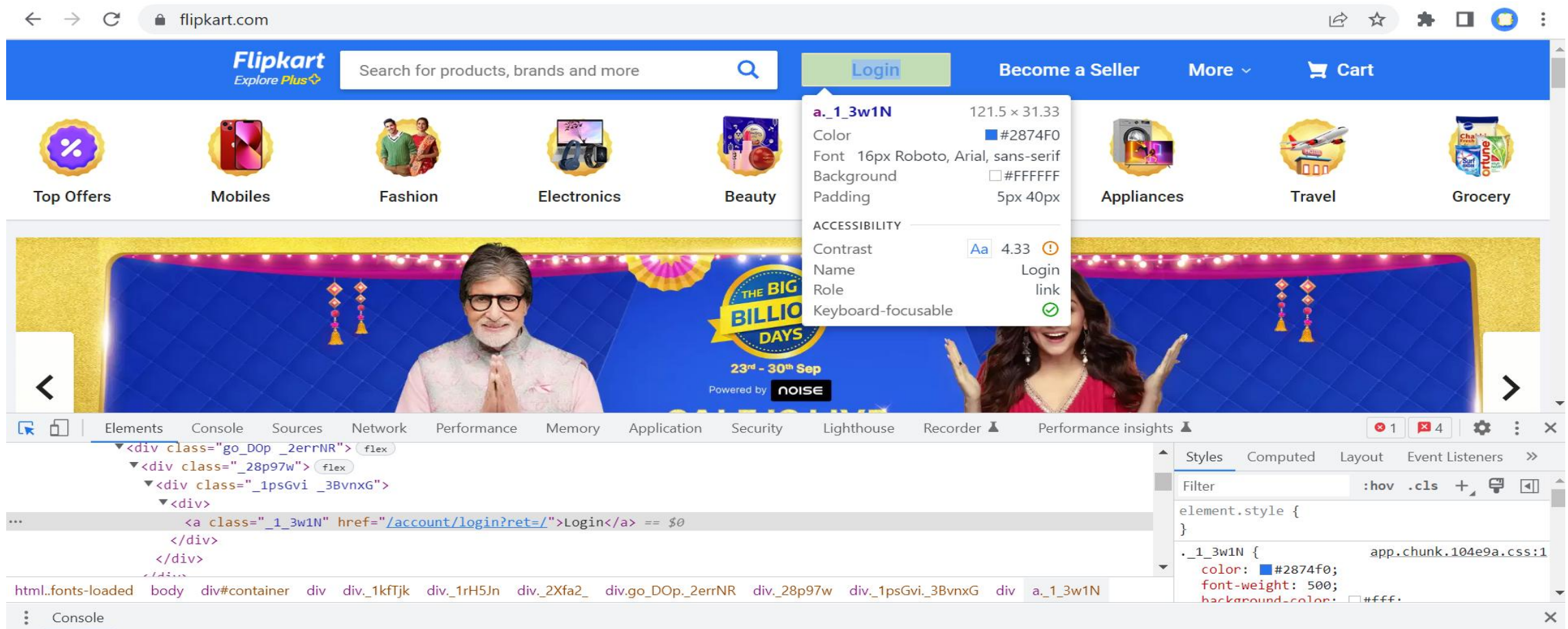
- Launch google chrome browser and navigate to flipkart website. You can see login.
- Now will locate the login with the help of LinkText Locator.



Locators

LinkText Example

- You will notice that the login is in anchor tag with hyperlink and link text = “Login”.



The screenshot shows the Flipkart homepage. At the top, there's a blue navigation bar with the Flipkart logo, a search bar, and links for 'Login', 'Become a Seller', 'More', and 'Cart'. Below this is a category bar with icons for Top Offers, Mobiles, Fashion, Electronics, Beauty, Appliances, Travel, and Grocery. A large banner for 'THE BIG BILLION DAYS' is in the center, featuring a man in a pink shirt and a woman in a red dress. A context menu is open over the 'Login' link, displaying its properties: Name (Login), Role (link), and Keyboard-focusable (checked). The menu also shows the element's dimensions (121.5 x 31.33), color (#2874F0), font (16px Roboto, Arial, sans-serif), background (#FFFFFF), and padding (5px 40px). At the bottom, the browser's developer tools are open, showing the HTML structure and the CSS styles for the selected element.

Flipkart
Explore Plus

Search for products, brands and more

Login

Become a Seller

More

Cart

Top Offers

Mobiles

Fashion

Electronics

Beauty

Appliances

Travel

Grocery

THE BIG BILLION DAYS

23rd - 30th Sep

Powered by NOISE

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Performance insights

```

<div class="go_DOp_2errNR">
  <div class="_28p97w">
    <div class="_1psGvi_3BvnxG">
      <div>
        <a class="_1_3w1N" href="/account/login?ret="/>Login</a> == $0
      </div>
    </div>
  </div>
</div>

```

html..fonts-loaded body div#container div div_1kftjk div_1rH5Jn div_2Xfa2 div.go_DOp_2errNR div_28p97w div_1psGvi_3BvnxG div a_1_3w1N

Console

Styles Computed Layout Event Listeners

Filter :hov .cls

```

element.style {
}

._1_3w1N {
  color: #2874f0;
  font-weight: 500;
  background-color: #fff;
}

```

C# Selenium Program – LinkText Locator

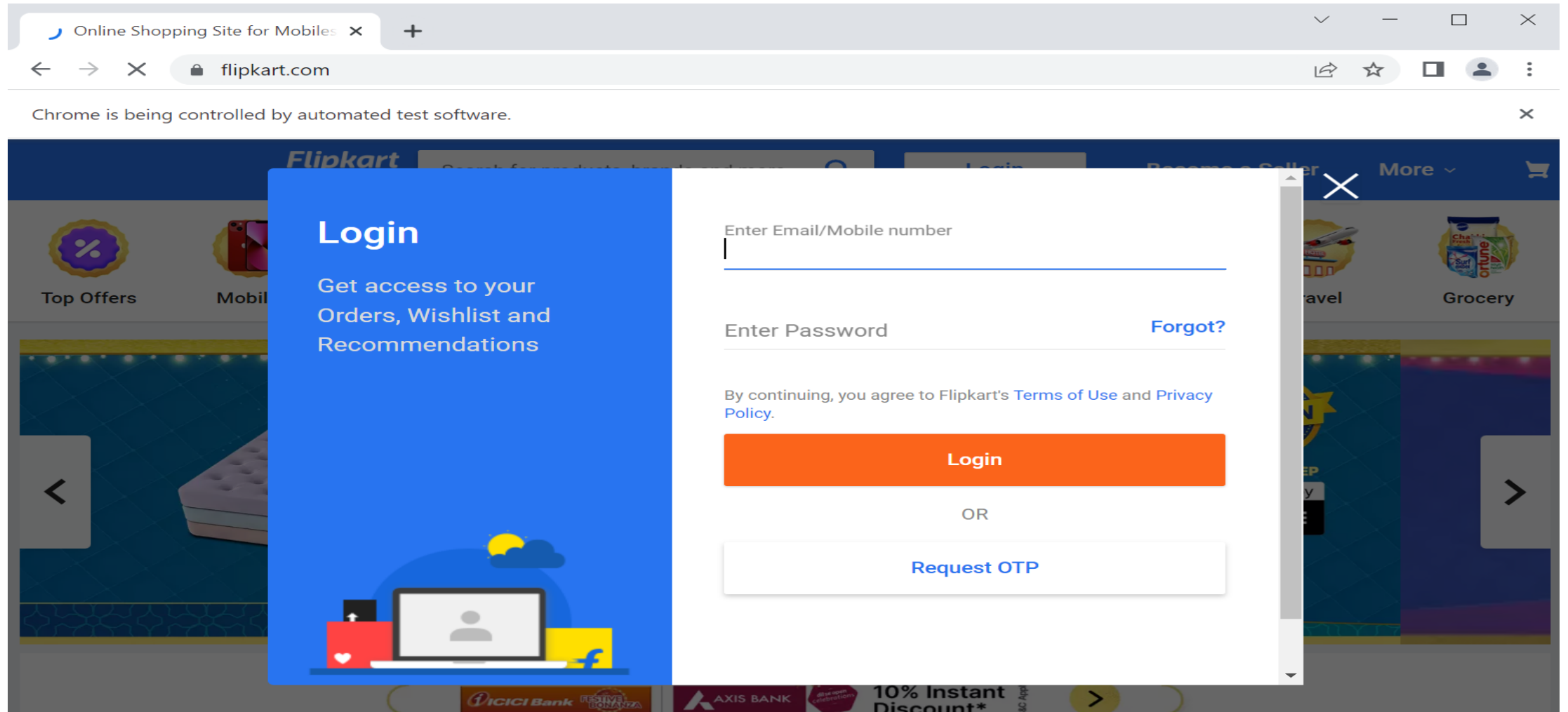
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.flipkart.com/")

#5. Using Link Text Locator
driver.find_element(By.LINK_TEXT, value="Login").click()

time.sleep(3)
```

C# Selenium Program – LinkText Locator - Output



Locators

PartialLinkText

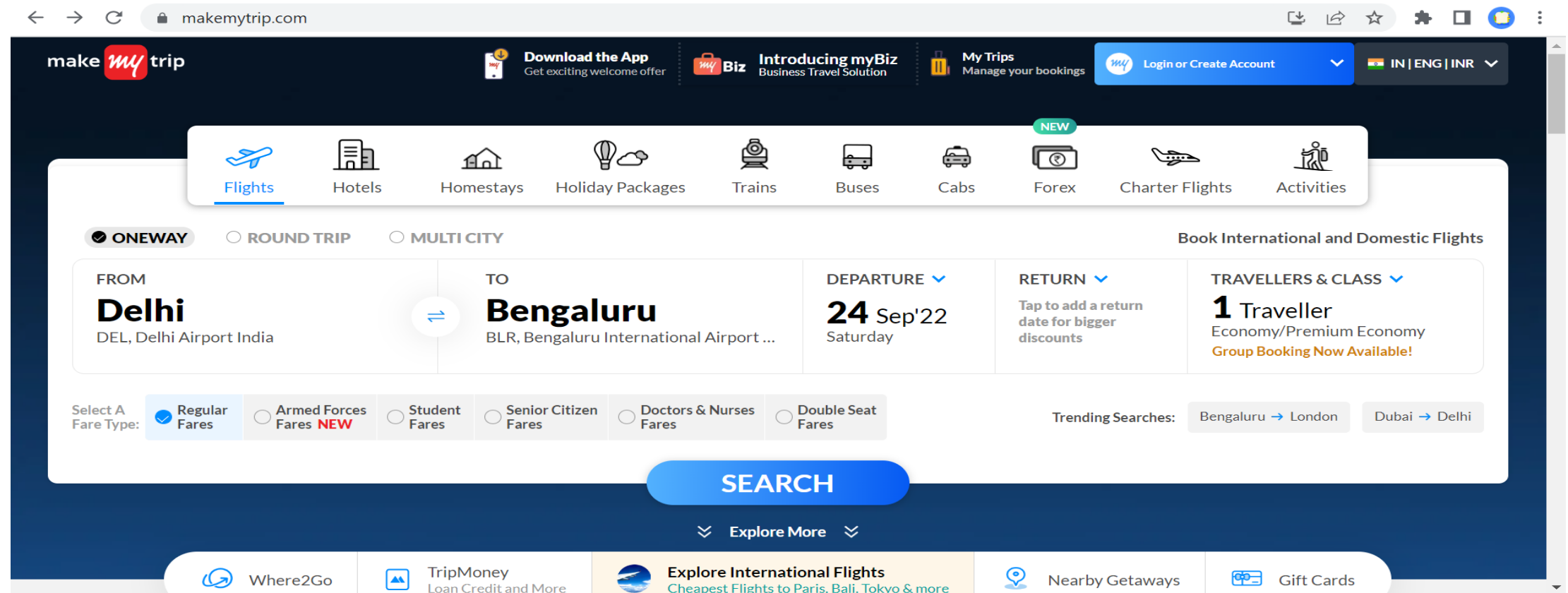
- This locator locates an element by a partial match of its link text and then performs actions on it. It works in the same way as the link text.
- We can use this technique to use only one piece of text that should be an anchor tag to locate a web element.
- The general syntax to use id locator is as follows:

Element = driver.find_element(By.PARTIAL_LINK_TEXT, value="Attribute value")

Locators

PartialLinkText Example

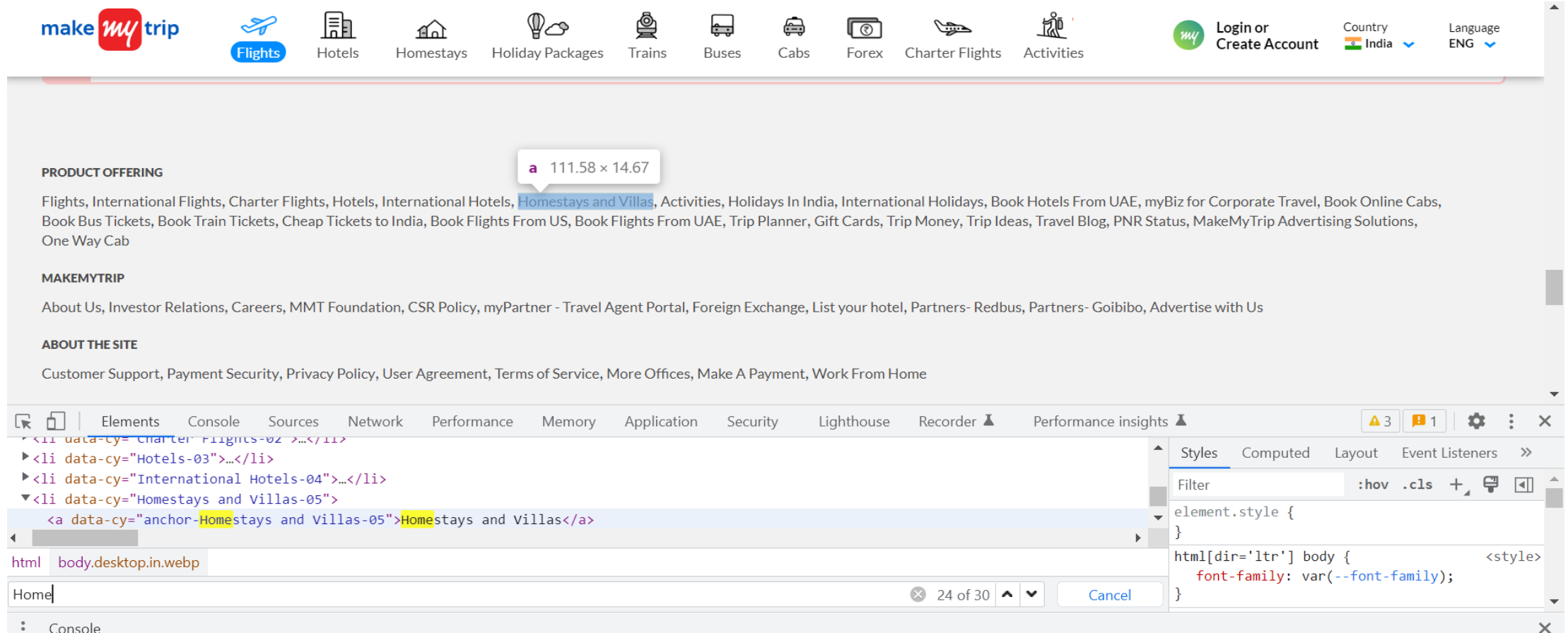
- Launch google chrome browser and navigate to makemytrip website. You can see Homestays.
- Now will locate the Homestays with the help of PartialLinkText Locator.



Locators

PartialLinkText Example

- You will notice that the Homestays is in anchor tag with hyperlink and link text = “Login”.



The screenshot shows the MakeMyTrip website header with navigation links: Flights, Hotels, Homestays, Holiday Packages, Trains, Buses, Cabs, Forex, Charter Flights, and Activities. A 'Login or Create Account' button is visible. The main content area lists various travel services under 'PRODUCT OFFERING'. The browser's developer tool is open, showing the HTML structure of the 'Homestays and Villas' link. The selected element is an anchor tag with the text 'Homestays and Villas'.

PRODUCT OFFERING

Flights, International Flights, Charter Flights, Hotels, International Hotels, Homestays and Villas, Activities, Holidays In India, International Holidays, Book Hotels From UAE, myBiz for Corporate Travel, Book Online Cabs, Book Bus Tickets, Book Train Tickets, Cheap Tickets to India, Book Flights From US, Book Flights From UAE, Trip Planner, Gift Cards, Trip Money, Trip Ideas, Travel Blog, PNR Status, MakeMyTrip Advertising Solutions, One Way Cab

MAKEMYTRIP

About Us, Investor Relations, Careers, MMT Foundation, CSR Policy, myPartner - Travel Agent Portal, Foreign Exchange, List your hotel, Partners- Redbus, Partners- Goibibo, Advertise with Us

ABOUT THE SITE

Customer Support, Payment Security, Privacy Policy, User Agreement, Terms of Service, More Offices, Make A Payment, Work From Home

HTML Structure (Developer Tool):

```

<li data-cy="Charter Flights-02">...</li>
<li data-cy="Hotels-03">...</li>
<li data-cy="International Hotels-04">...</li>
<li data-cy="Homestays and Villas-05">
  <a data-cy="anchor-Homestays and Villas-05">Homestays and Villas</a>
</li>
  
```

Styles Panel:

```

element.style {
}

html[dir='ltr'] body {
  font-family: var(--font-family);
}
  
```

C# Selenium Program – PartialLinkText Locator

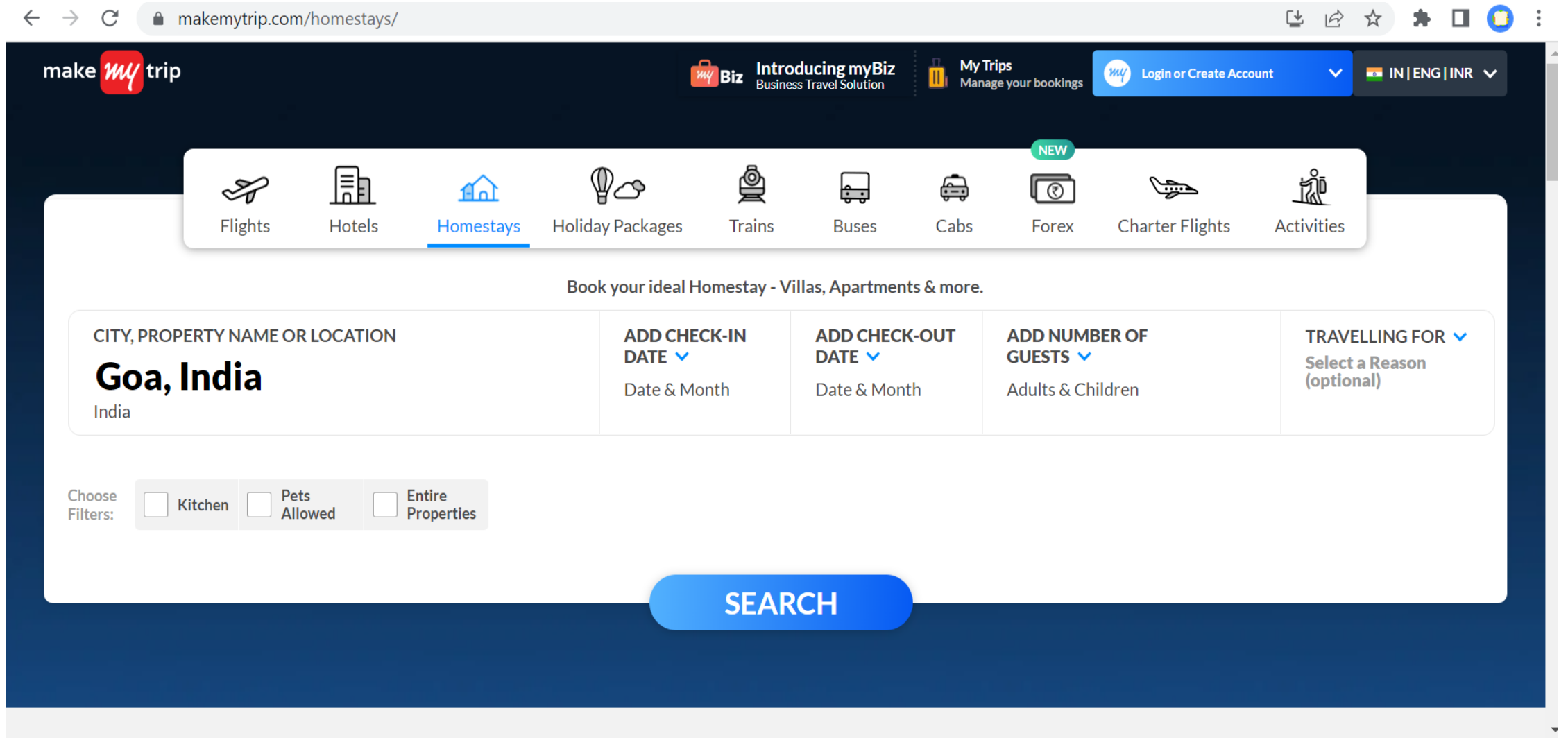
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.makemytrip.com/")

#6. Using Partial Link Text Locator
driver.find_element(By.PARTIAL_LINK_TEXT, value="Home").click()

time.sleep(3)
```

C# Selenium Program – PartialLinkText Locator - Output



The screenshot shows the 'makemytrip' website's 'homestays' page. The browser's address bar displays 'makemytrip.com/homestays/'. The page features a navigation bar with icons for Flights, Hotels, Homestays (which is underlined and highlighted), Holiday Packages, Trains, Buses, Cabs, Forex, Charter Flights, and Activities. Below the navigation bar, a search form is visible with the following fields:

- CITY, PROPERTY NAME OR LOCATION:** Contains 'Goa, India' with 'India' as a suggestion below it.
- ADD CHECK-IN DATE:** Includes a dropdown arrow and the text 'Date & Month'.
- ADD CHECK-OUT DATE:** Includes a dropdown arrow and the text 'Date & Month'.
- ADD NUMBER OF GUESTS:** Includes a dropdown arrow and the text 'Adults & Children'.
- TRAVELLING FOR:** Includes a dropdown arrow and the text 'Select a Reason (optional)'.

Below the search form, there are filter options under the heading 'Choose Filters:':

- ☐ Kitchen
- ☐ Pets Allowed
- ☐ Entire Properties

A large blue 'SEARCH' button is positioned at the bottom of the search form.

Types of Locators

Xpath

Locator – Xpath

- XPath stands for XML Path.
- It's a query language that helps identify elements from an XML document.
- It uses expressions that navigate into an XML document in a way that can be traced from the start to the intended element like forming a path from the start



Xpath

Types of XPath

Xpath Types

- Absolute Xpath
- Relative Xpath
- The node is selected by following a path or steps. We can use XPath to find the location of any element on a webpage.



Selecting Nodes in XPath:

Expression	Description
/	Selects from the root node
//	Selects elements relative to reference node that match the selection anywhere inside the HTML DOM
@	Selects attributes

Xpath

Absolute Xpath

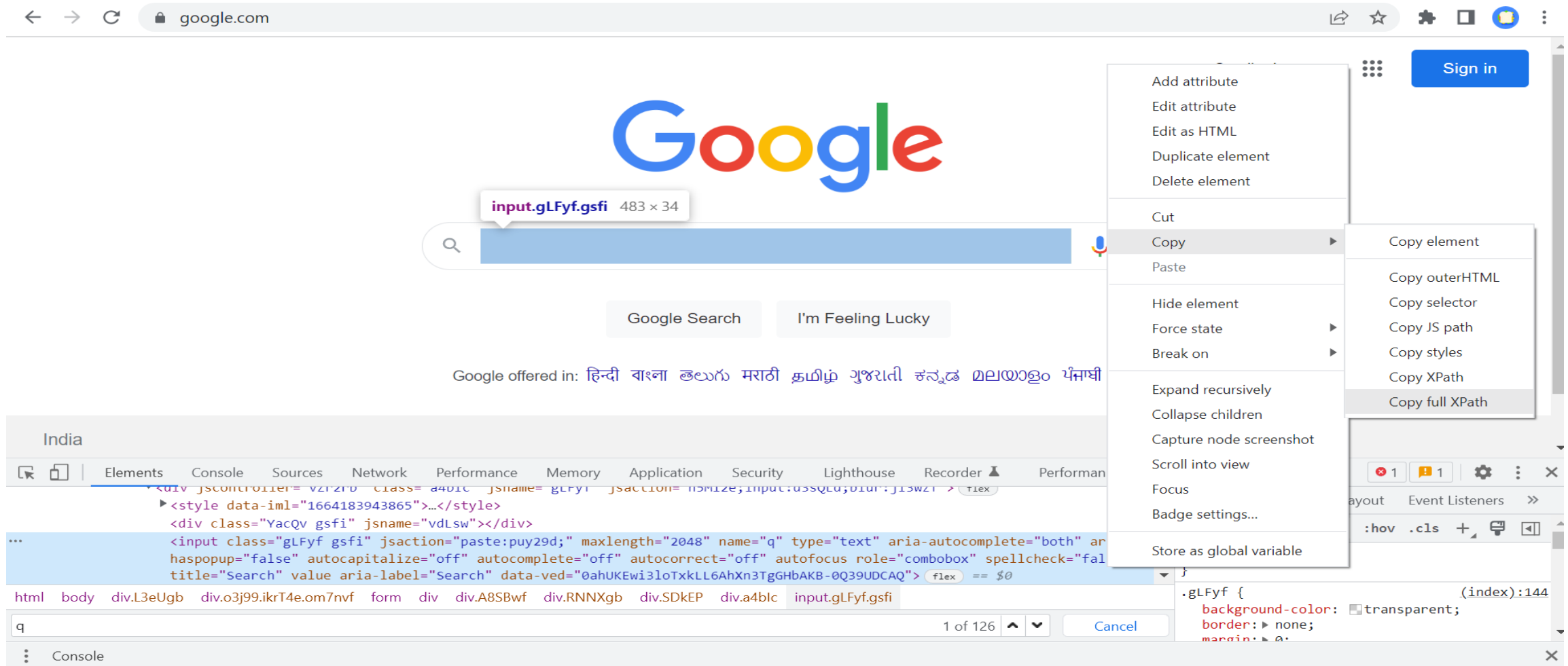
- Absolute xpath has the complete path beginning from the root to the element which we want to identify.
- An absolute xpath starts with the / symbol.
- An absolute xpath is lengthy and difficult to maintain.
- if any changes are made in the path of the element then this XPath gets failed. So, This is the main disadvantage of absolute XPath.

Syntax:

- `Element = driver.find_element(By.XPATH, value="/html/body/div[1]/div[3]/form/div[1]/div[1]/div[1]/div/div[2]/input")`

Xpath

Absolute Xpath



The screenshot shows the Google homepage with the developer tools open. The search input field is selected, and a tooltip displays its XPath: `input.gLFyf.gsfi` with dimensions 483 x 34. The developer tools pane shows the HTML structure of the search bar, and the XPath console displays the absolute XPath: `html/body/div.L3eUgb/div.o3j99.ikrT4e.om7nvf/form/div/div.A8SBwf/div.RNNXgb/div.SDkEP/div.a4blc/input.gLFyf.gsfi`.

Google Search I'm Feeling Lucky

Google offered in: हिन्दी বাংলা తెలుగు मराठी தமிழ் ગુજરાતી ಕನ್ನಡ മലയാളം ਪੰਜਾਬੀ

India

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Performance

```

<div jscontroller="vZr2rD" class="a4v1C" jsname="gLFyf" jsaction="paste:puy29d;" maxlength="2048" name="q" type="text" aria-autocomplete="both" ar
  <div class="YacQv gsfi" jsname="vdLsw"></div>
  <input class="gLFyf gsfi" jsaction="paste:puy29d;" maxlength="2048" name="q" type="text" aria-autocomplete="both" ar
    haspopup="false" autocapitalize="off" autocomplete="off" autocorrect="off" autofocus role="combobox" spellcheck="fal
    title="Search" value aria-label="Search" data-ved="0ahUKEwi3loTxkLL6AhXn3TgGHbAKB-0Q39UDCAQ">
  
```

html body div.L3eUgb div.o3j99.ikrT4e.om7nvf form div div.A8SBwf div.RNNXgb div.SDkEP div.a4blc input.gLFyf.gsfi

q 1 of 126 Cancel

Console

gLFyf { (index):144
background-color: transparent;
border: none;
margin: 0;

C# Selenium Program – Absolute Xpath Locator

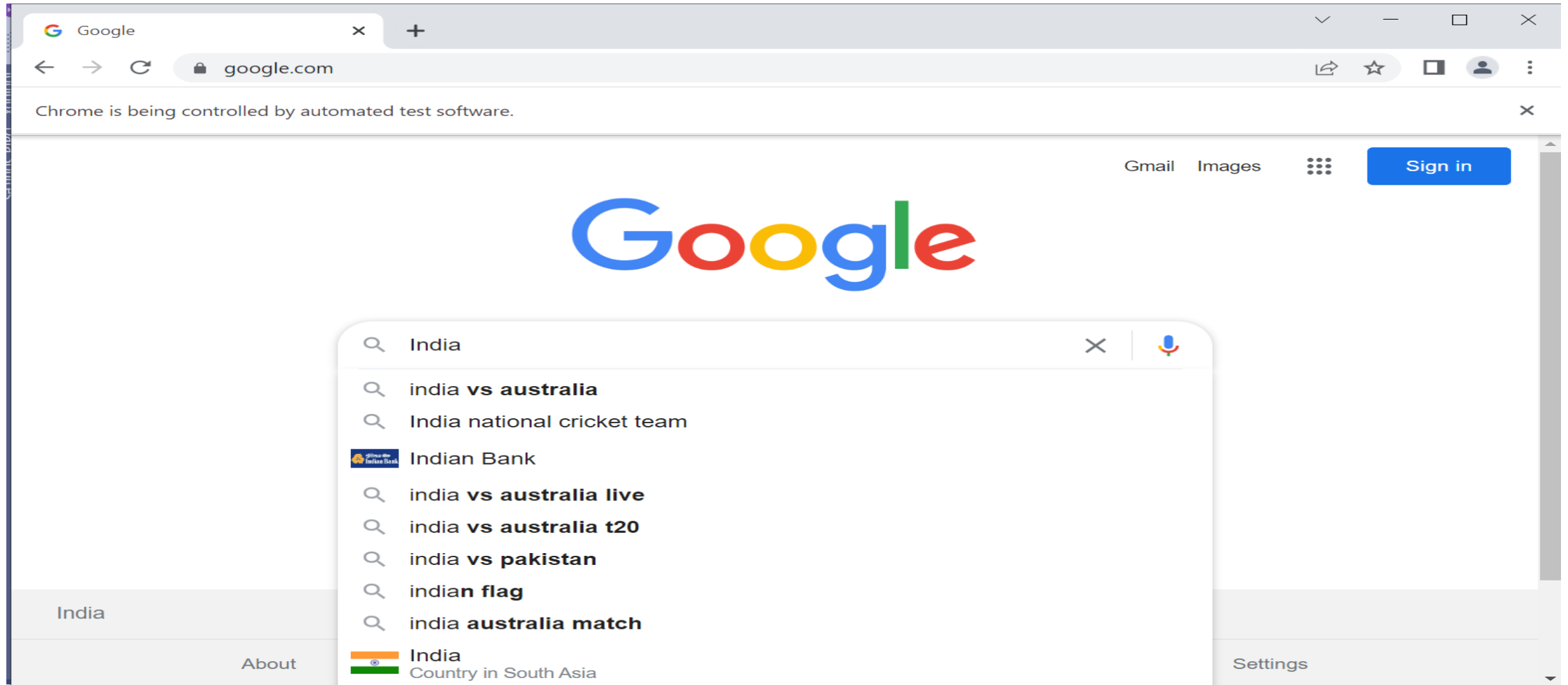
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.google.com/")

#7. Using Absolute XPath Locator
driver.find_element(By.XPATH, value=("/html/body/div[1]/div[3]/form/div[1]/div[1]/div[1]/div/div[2]/input ").click()

time.sleep(3)
```

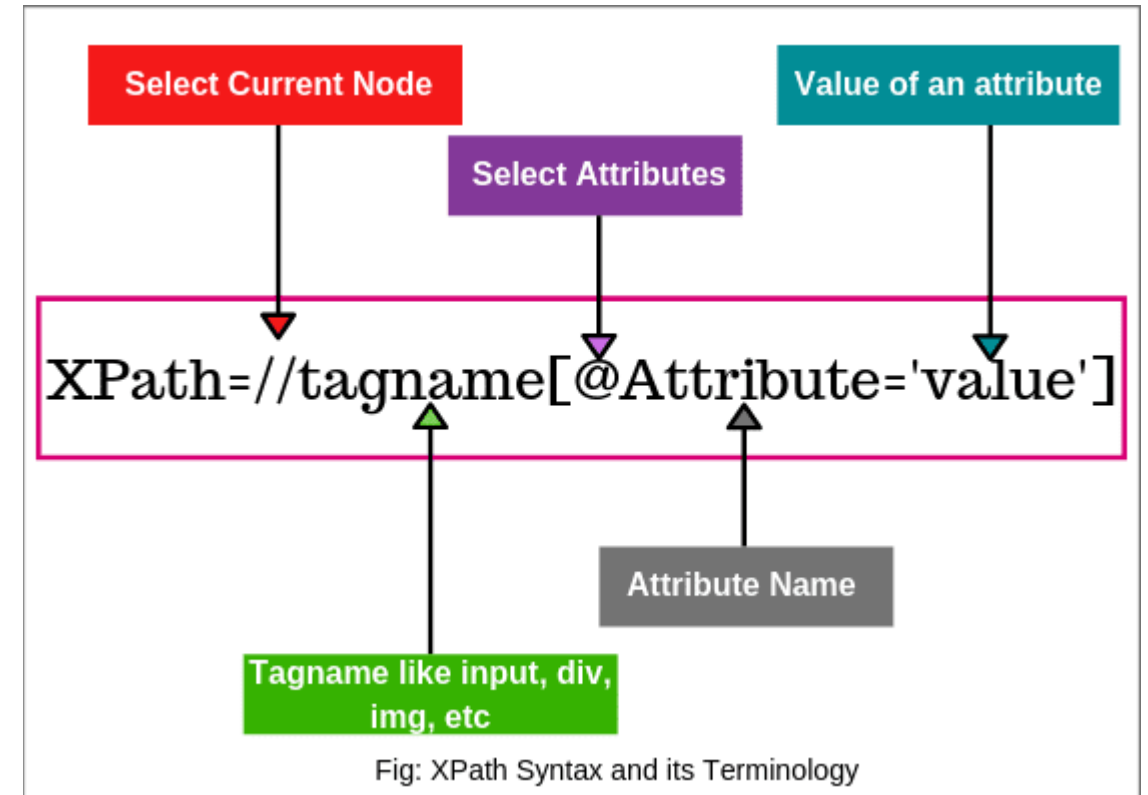
C# Selenium Program – Absolute Xpath Locator - Output



Xpath

Relative Xpath

- The relative xpath starts by referring to the element that we want to identify and not from the root node.
- A relative xpath starts with the “//”symbol
- Even if an element is removed or added in the DOM, the relative xpath is not impacted
- It can search elements anywhere on the webpage



C# Selenium Program – Relative Xpath Locator

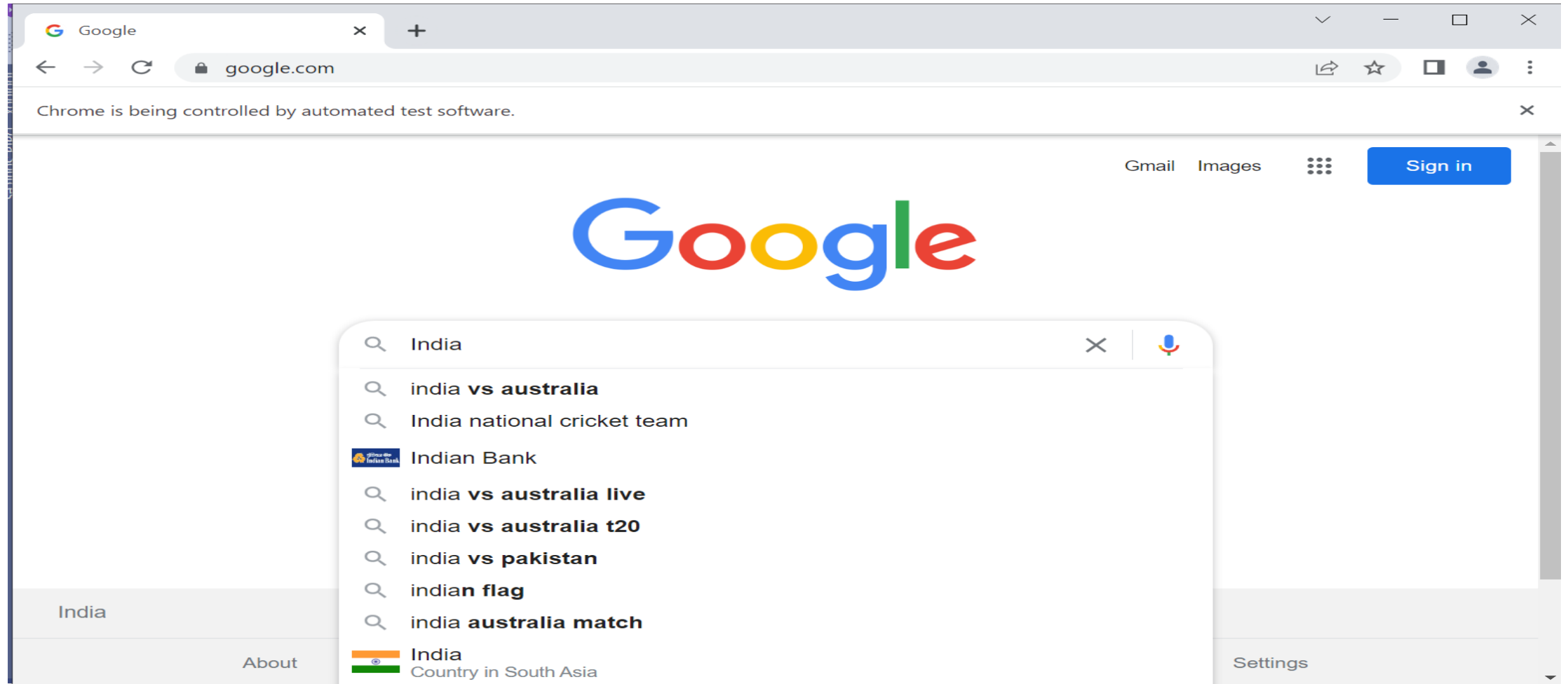
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.google.com/")

#7. Using Relative XPath Locator
driver.find_element(By.XPATH, value="//*[@name='q']").send_keys("India")

time.sleep(3)
```

C# Selenium Program – Absolute Xpath Locator - Output



Xpath

Xpath - Contains

- contains() is a function within Xpath expression which is used to search for the web elements that contain a particular text
- We can extract all the elements that match the given text value using the XPath contains() function throughout the webpage
- Contains in XPath has ability to find the element with partial text

Syntax :

```
//<HTML tag>[contains(@attribute_name,'attribute_value')]
```


Xpath

XPath - Starts-with

- starts-with() method is used when we know about the initial partial attribute value or initial partial text associated with the web element.
- User can also use this method to locate web elements those consists of both static(initial) and dynamic(trailing) values.

Syntax:

XPath: //tagname[starts-with(@attribute, 'value')]

Example:

- //input[starts-with(@placeholder, 'Organization')]
- //input[starts-with(@name, 'organization')]

Xpath

XPath - Ends-with

- ends-with() method checks the ending text of an attribute and finds elements whose attribute changes dynamically on refresh.

Syntax:

XPath: //tagname[ends-with(@attribute, 'value')]

Example:

```
//input[ends-with(@placeholder, 'Organization')]
```

```
//input[ends-with(@name, 'organization')]
```

Xpath

XPath - “OR” Statement

- In OR expression, two conditions are used, whether 1st condition OR 2nd condition should be true.
- It is also applicable if any one of the condition is true or maybe both.
- Means any one condition should be true to find the element.

Syntax:

XPath: //tagname[XPath statement-1 or XPath statement-2]

Example:

```
//input[@value = 'Log In' or @type = 'submit']
```

Xpath

XPath - “AND” Statement

- In AND expression, two conditions are used, both conditions should be true to find the element
- It fails to find the element if any one condition is false.

Syntax:

XPath: //tagname[XPath statement-1 and XPath statement-2]

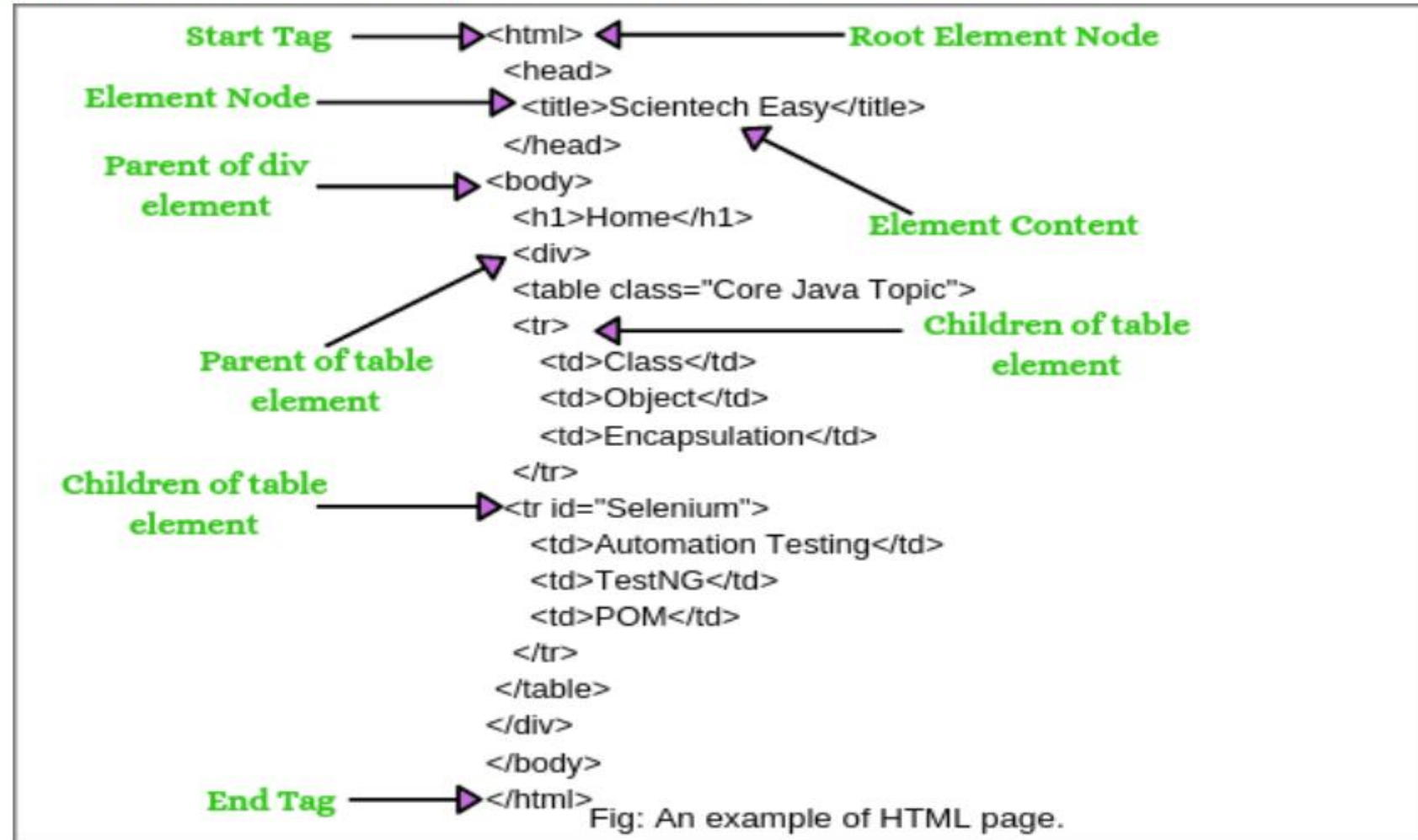
Example:

```
//input[@value = 'Log In' and @type = 'submit']
```

Xpath

XPath – Axes

An example of a basic HTML page is shown below screenshot.



Xpath

XPath – Parent

- Parent in Selenium is a method used to retrieve the parent node of the current node selected in the web page
- It is very useful in the situation when you select an element and need to get the parent element using Xpath.

Example:

```
Xpath=//*[@id='rt-feature']//parent::div
```

```
Xpath=//*[@title="50"]//parent::store
```

Xpath

Xpath – Ancestor

- The ancestor axis selects all ancestor's element (parent, grandparent, great-grandparents, etc.) of the current node.
- This axis always contains the root node (unless the current node is the root node).

Syntax:

```
//ancestor::tagName
```

Xpath – Ancestor (Contd.)

Example:

- Let us consider the login button as current node. First, find the Xpath of current node.

XPath(Current node): //input[@id = 'u_0_a']

- Now, we will find XPath of parent and grandparent of current node.

XPath(Parent node): //input[@id = 'u_0_a']//ancestor::label

XPath(Grandparent node): //input[@id = 'u_0_a']//ancestor::td

- In both cases, 1 of 1 node is matched by using “ancestor” axis.

XPath – Sibling

- A Sibling in Selenium Webdriver is a function used to fetch a web element which is a sibling to the parent element.
- If the parent element is known then the web element can be easily found or located that can use the sibling attribute of the Xpath expression in selenium webdriver

Example:

- 1) `//div[@class='canvas- graph'] //a[@href ='/ accounting.html'] [i[@class='icon-
usd']] /following-sibling::h4 Xpath=//*[title="50"]/parent::store`
- 2) `//input[@id = 'u_0_5'] /following-sibling::label`

XPath – Sibling

Example:

```
<ul class="right">
  <li><a href="https://www.educba.com/feature">Home</a></li>
  <li><a href="https://www.educba.com/Blogs">Automation</a></li>
  <li><a href="https://www.educba.com/product">Product</a></li>
  <li><a href="https://www.educba.com/pricing">Pricing</a></li>
  <li><a href="https://www.educba.com/support/">Support</a></li>
  <li class="signout"><a href="https://accounts.educba.com/login">Login</a></li>
  <li class="login"><a href="https://accounts.educba.com/login">FreeSignin</a>      </li>

//li[@class='sign-out']//following-sibling::li“

//li[@class='sign-out']//preceding-sibling::li"
```

Xpath

Advantages of Xpath

- Queries are easy to type and read.
- Query strings are easily embedded in programs, scripts, and XML or HTML attributes.
- Queries are easily parsed.
- You can specify any path that can occur in an XML document and any set of conditions for the nodes in the path.
- You can uniquely identify any node in an XML document
- Queries return any number of results, including zero.
- Query conditions can be evaluated at any level of a document and are not expected to navigate from the top node of a document.
- Queries do not return repeated nodes

Xpath

Advantages of Xpath (Contd.)

- For programmers, queries are declarative, not procedural. They say what should be found, not how it should be found. This is important because a query optimizer must be free to use indexes or other structures to find results efficiently.
- XPath is designed to be used in many contexts. It is applicable to providing links to nodes, for searching repositories, and for many other applications

CSS Selector

CSS Selector

- CSS Selectors in Selenium are string patterns used to identify an element based on a combination of HTML tag, id, class, and attributes. CSS Selectors in Selenium have many formats, but we will only focus on the most common ones.
- `css = element_name[<attribute_name>=<value>']`

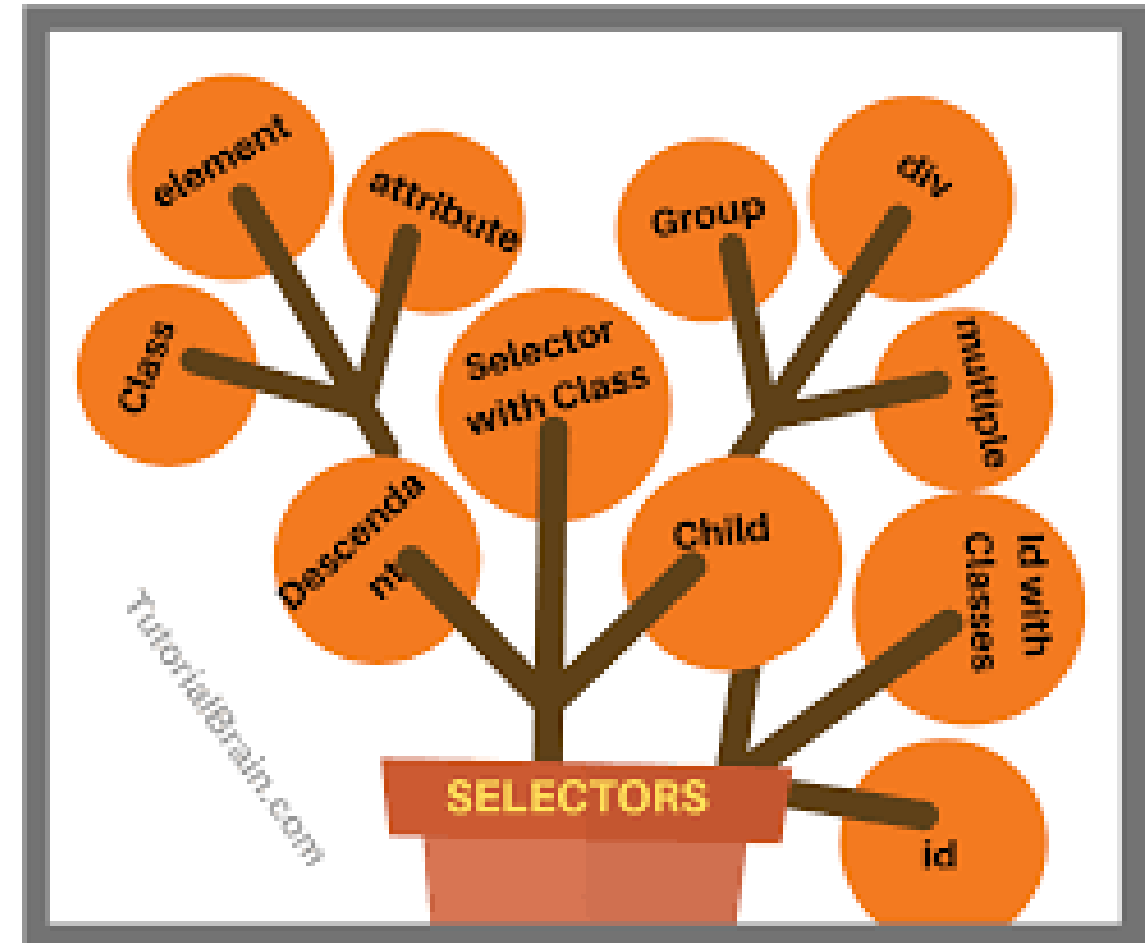
Example:

```
Element = driver.find_element(By.CSS_SELECTOR, value="input[name='first_name']")
```

CSS Selector

CSS Selector (Contd.)

- Tag and ID
- Tag and class
- Tag and attribute
- Tag, class, and attribute
- Inner text



CSS Selector

CSS Selector (Contd.)

- Tag and ID
 - To locate elements by Tag and ID, you have to use the following components:
 - Html tag: It provides the tag we wish to locate (e.g. input tag).
 - #: It is used to represent the ID attribute.
 - Keep in mind that when you wish to locate an element via ID through a CSS selector, it must have a hash sign on the same. For other attributes, we need not use the hash sign.
 - Value of the ID attribute: This represents the value of the ID we are using to locate the element.
 - Syntax: `css=(Html tag )(#) (value of the ID attribute)`
 - Example: `driver.find_element(By.CSS_SELECTOR, value="input#userpassword")`

CSS Selector

CSS Selector (Contd.)

- Tag and Class

- Syntax:

`css=(HTML tag)(.)(Value of Class attribute)`

- Example:

- `driver.find_element(By.CSS_SELECTOR, value="button.submit-btn")`

CSS Selector

CSS Selector (Contd.)

- Tag, Class and Attribute

- Syntax:

css=(HTML tag>)(.)(Class attribute value)([attribute=Value of attribute])

- Example: `driver.find_element(By.CSS_SELECTOR, value="button.submit-btn[data-callback=\"onSubmit\"]")`
- Here is the DOM structure:

```
<button data-sitekey="6LceAqQaAAAAAO0LclgLnXy3gH_M3X5aDrqUihHw" data-callback="onSubmit" data-amplitude="R_signup" type="submit" class="btn btn-dark submit-btn g-recaptcha" css="1">Free Sign Up</button>
```

CSS Selector

Wild (*, ^ and \$) in CSS for classes

- Selenium CSS selector in Selenium helps in matching multiple strings through the use of multiple patterns like ^, \$, *.
- Wildcard selectors in CSS are used for selecting multiple elements simultaneously.

a. Starts-With in CSS Selector

- The Starts-With helps locate elements when we try to match elements with a string that starts with a designated value.
- **Syntax** `css=(HTML tag)([attribute^=start of the string])`

CSS Selector

Wild (*, ^ and \$) in CSS for classes

Example

- Here is the DOM structure for locating the WebElement:

```
<input type="email" name="email" value="" placeholder="Email" required="required"
      autofocus="autofocus" class="form-control mt-3 form-control-lg">
```

- Here how the CSS [attribute^=value] Selector is used for locating the desired WebElement:

```
driver.find_element(By.CSS_SELECTOR, value="input[name^='em']")
```

CSS Selector

Wild (*, ^ and \$) in CSS for classes

b. Ends-With in CSS Selector

- This helps locate elements when we try to match elements with a string that ends with a designated value.

- **Syntax** `css=(HTML tag)([attribute$=end of the string])`

- **Example**

- Here is the DOM structure of the element:

```
<input type="email" name="email" value="" placeholder="Email" required="required"
autofocus="autofocus" class="form-control mt-3 form-control-lg">
```

- Here is how Ends-With in CSS Selector is used for locating the required WebElement:

```
driver.find_element(By.CSS_SELECTOR, value="input[name$='ail']")
```

CSS Selector

Wild (*, ^ and \$) in CSS for classes

c. Contains in CSS Selector

- This helps locate elements when we try to match elements with a string that containing a designated value.

- **Syntax** `css=(HTML tag)([attribute*=partial string])`

- **Example**

- Here is the DOM structure of the element:

```
<input type="email" name="email" value="" placeholder="Email" required="required"
autofocus="autofocus" class="form-control mt-3 form-control-lg">
```

- Here is how Contains in CSS Selector is used for locating the required WebElement:

```
driver.find_element(By.CSS_SELECTOR, value="input[class*='control']")
```

CSS Selector

Advantages of CSS Selector

- It's faster than XPath.
- It's much easier to learn and implement
- You have a high chance of finding your elements.
- It's compatible with most browsers to date.

Disadvantages of CSS Selector

- The browser does not support CSS or the CSS selectors you need.
- Support one selector type.
 - 1) Traversing the DOM is not possible with CSS as that of the Xpath.
 - 2) Browser Compatibility issue. We have to determine which style is supported and which is not.
 - 3) Cross browser issue. Since some selectors behave differently in different browsers.

CSS Selector

Example Program 1

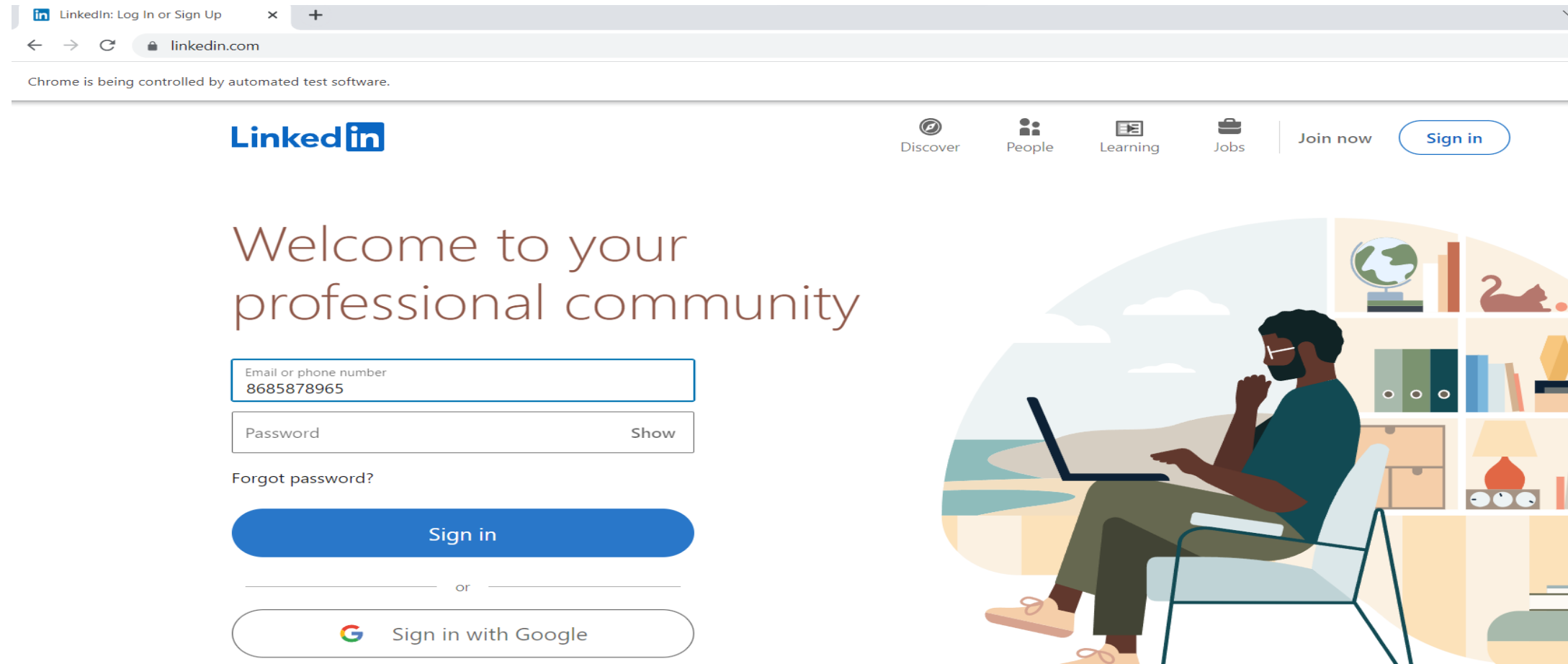
```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.linkedin.com/")

# Using CSS Selector Locator
driver.find_element(By.CSS_SELECTOR, value="input#session_key").send_keys("8685878965")

time.sleep(3)
```


Output



CSS Selector

Example Program 2

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.gmail.com/")

# Using CSS Selector Locator
driver.find_element(By.CSS_SELECTOR, value="input[id='yDmH0d']").send_keys("XYZ@gmail.com")

time.sleep(3)
```

Output

Chrome is being controlled by automated test software.


Sign in
to continue to Gmail

Email or phone

XYZ@gmail.com

[Forgot email?](#)

Not your computer? Use Guest mode to sign in privately.

[Learn more](#)

[Create account](#)

Next

English (United Kingdom) ▼

Help

Privacy

Terms

CSS Selector

Practice

- 1) Perform Automation Testing for <https://my.indiamart.com/> application using Java Selenium.
 - i)Launch the application in chrome browser
 - ii)Select the wheat Flour link
 - iii)Select Wheat grain
 - iv)Move to link Chennai and Select
 - v)Select and click the first product in resulting page
 - vi)Switch to the child window
 - vii)Get the latest price.
 - viii)Switch to parent window

Practice

- 1) Perform Automation Testing for <https://www.woodenstreet.com/> application using Java Selenium.
 - i)Launch the application in chrome browser
 - ii)Select the living storage link
 - iii)Select the TV Unit link
 - iv)Apply the filters
 - Price Under 9999
 - Honey Finish in Finish
 - v)Select the first option from the link
 - vi)Add it to the cart
 - vii)Move to the homepage
 - viii)Select the Study & Office
 - ix)Move to Office furniture
 - x)Select Workstation
 - xi) Apply Sort by Latest filter and with Storage in Storage.
 - xii)Add it to the cart.

THANK YOU